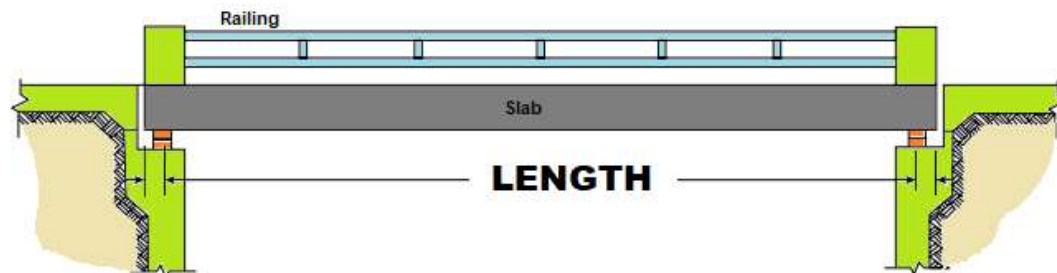


>> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<

COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A COMPOSITE BEAM SECTION SIMPLY SUPPORTED : AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION, STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS

THIS PROGRAM IS WRITTEN BY SALAR DELAVAR GHASHGHAEE (QASHQAI)



$$\text{Structure Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

$$\Delta_d = \text{Lateral Displaement from Dynamic Analysis}$$
$$\Delta_y = \text{Lateral Yield Displaement from Pushover Analysis}$$
$$\Delta_u = \text{Lateral Ultimate Displacement from Pushover Analysis}$$

$$P(t) = P_0 e^{-0.05\bar{\omega}t} \sin(\bar{\omega}t)$$

$$m\ddot{u} + c\dot{u} + ku = P(t)$$

$$\text{Structure Ductility Damage Index} = \frac{\Delta_d - \Delta_y}{\Delta_u - \Delta_y}$$

$$\rho A \frac{\partial^2 v(x,t)}{\partial t^2} + EI \frac{\partial^4 v(x,t)}{\partial x^4} = P(t)$$

boundary conditions

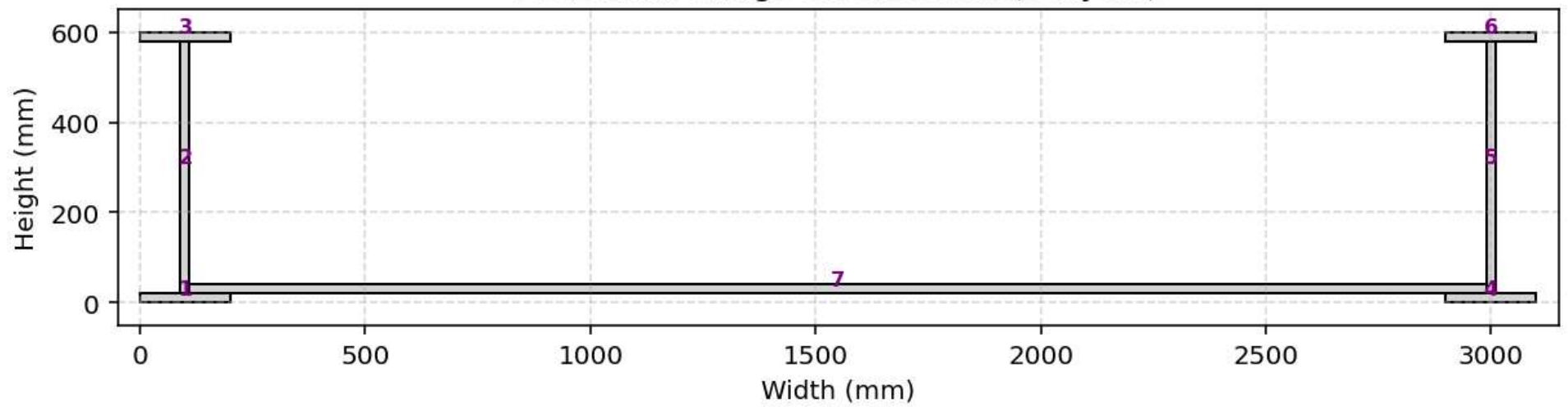
$$v(0,t) = v(L,t) = 0, \quad \left. \frac{\partial^2 v(x,t)}{\partial x^2} \right|_{x=0} = \left. \frac{\partial^2 v(x,t)}{\partial x^2} \right|_{x=L} = 0.$$







Pedestrian Bridge Steel section (7 layers)



Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

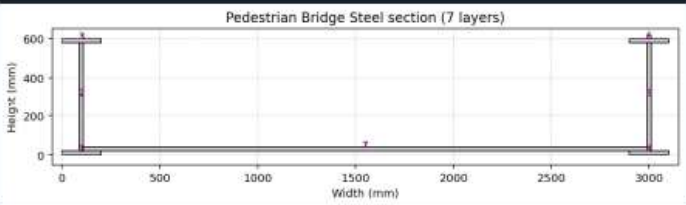
C:\Users\Dell\Desktop\OPENSEES_FILES\+HEIGHT_ANALY...IDGE_STEEL_SECTION_DIFF_WIDTH_HEIGHT_FUN_EXTRA.py

P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py X PEDESTRIAN_BRIDGE...HEIGHT_FUN_EXTRA.py X

```
1 def PEDESTRIAN_BRIDGE_STEEL_SECTION_DIFF_WIDTH_HEIGHT_FUN_EXTRA(secTag, STEEL_TYPE, STEEL_D
2 """
3 Create a steel with different widths and heights, fiber section (OpenSees) and,
4 if PLOT=True, draw a simple 2-D sketch of the section.
5
6 Parameters
7 -----
8 secTag      : int - section identifier
9 STEEL_TYPE   : str - 'ELASTIC' or 'INELASTIC'
10 STEEL_DENSITY: float - pc (kg/m³) - will be converted to N·s²/mm³
11 PLOT        : bool - draw the section if True
12
13 THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
14 """
15 import numpy as np
16 import openseespy.opensees as ops
17 import matplotlib.pyplot as plt
18 import matplotlib.pyplot as plt, matplotlib.patches as patches
19
20 # Define materials for nonlinear elements
21 # Define parameters (units: mm, N)
22 # -----
23 # STEEL
24 fy = 240      # [N/mm²] Steel Yield Strength
25 Es = 2e5      # [N/mm²] Modulus of Elasticity
26 ey = fy/Es    # [mm/mm] Steel Yield Strain
27 fu = 1.1818*fy # [N/mm²] Steel Ultimate Strength
28 esu = 0.15    # [mm/mm] Steel Ultimate Strain
29 Esh = (fu - fy)/(esu - ey)
30 Bs = Esh / Es
31
32 steelTag = secTag + 100
33 if STEEL_TYPE == 'ELASTIC':
34     ops.uniaxialMaterial('Elastic', steelTag, Es)
```

36 %

Pedestrian Bridge Steel section (7 layers)



IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 1, Col 1 UTF-8 CRLF RW Mem 45%

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py X PEDESTRIAN_BRIDGE_...EIGHT_FUN_EXTRA.py X

```
1 #####
2 # >> IN THE NAME OF ALLAH, THE MOST GRACIOUS, THE MOST MERCIFUL <<
3 # COMPREHENSIVE NONLINEAR SEISMIC ASSESSMENT OF A PEDESTRIAN BRIDGE STEEL I SECT
4 # AN OPENSEES FRAMEWORK FOR STATIC PUSHOVER, CYCLIC DEGRADATION,
5 # STATIC TIME-HISTORY AND DYNAMIC TIME-HISTORY ANALYSIS
6 #-----
7 # P(t) = P0 sin(wt)
8 # P(t) = P0 exp(-0.05wt) sin(wt)
9 #-----
10 # ASSESSMENT OF DUCTILITY DAMAGE INDICES FOR STRUCTURAL ELEMENTS AND SYSTEMS AND E
11 # OF ENERGY DISSIPATION CAPACITY INDEX
12 #-----
13 # THIS PYTHON SCRIPT WRITTEN BY SALAR DELAVAR GHASHGHAEI (QASHQAI)
14 # EMAIL: salar.d.ghashghaei@gmail.com
15 #####
16 """
17 Nonlinear Seismic Performance Assessment of a pedestrain bridge steel I section:
18 An OpenSeesPy Framework for Material and Geometric Nonlinearity Under Static, Cyclic, and
19
20 This OpenSeesPy script performs rigorous nonlinear static and dynamic analysis of a pedest
21 for performance-based earthquake engineering. The 2D model incorporates both material non
22 (elastic-perfectly plastic or hysteretic steel with strain hardening)
23 and geometric nonlinearity to capture P-delta effects
24 and large displacements-essential for collapse assessment.
25
26 Eight analysis protocols are implemented:
27 (1) [PERIOD] : Structural Period
28 (2) [STATIC] : Gravity Load analysis establishing dead Load state
29 (3) [PUSHOVER] : Displacement-controlled pushover generating full capacity curves
30 and plastic mechanism identification
31 (4) [CYCLIC_DISPLACEMENT] : Symmetric cyclic displacement protocol capturing hysteresis,
32 pinching behavior, and energy dissipation degradation
33 (5) [STATIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Static Analysis of External time-dependent
34 (6) [DYNAMIC_EXTERNAL_TIME-DEPENDENT_LOADING] : Dynamic Analysis of External time-dependen
```

36 %

Pedestrian Bridge Steel section (7 layers)

Height (mm)

Width (mm)

IPython Console Files Help Variable Explorer Debugger Plots History

IntelliJ Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 75 Col 2 LITE-8 CRLF RW Mem 45%

STATIC ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

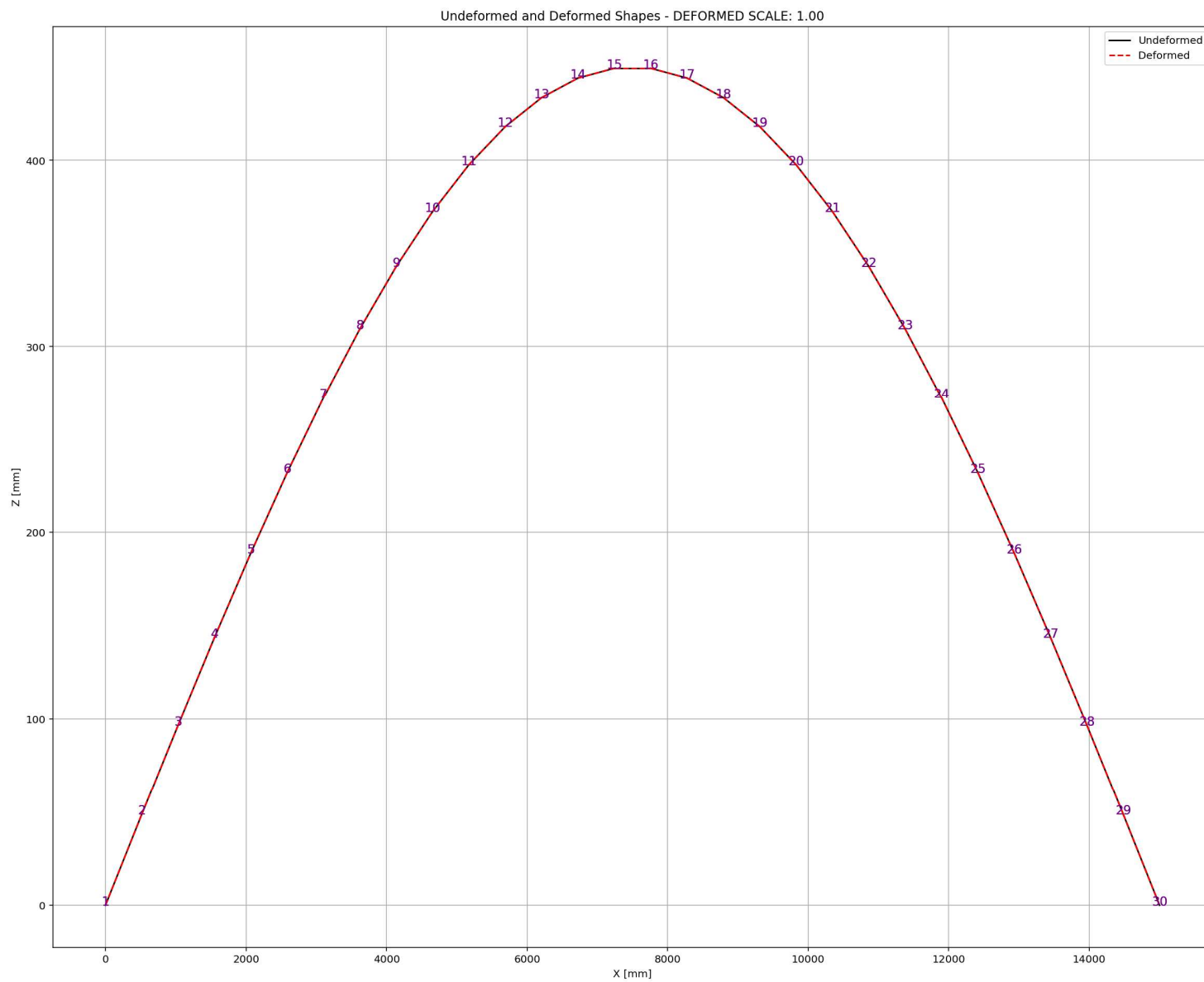
P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py x PEDESTRIAN_BRIDGE...EIGHT_FUN_EXTRA.py x

```
251
252 if ANAL_TYPE == 'STATIC':
253     ops.integrator('LoadControl', 1.0)
254     ops.analysis('Static')
255     OK = ops.analyze(1)
256     S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
257     ops.reactions()
258     reaction.append(ops.nodeReaction(1, 2)+ops.nodeReaction(30, 2)) # SHEAR BASE REACT
259     disp_mid.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT NOD
260     dispX.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT NOD
261     dispY.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT NOD
262     # Store axial forces, strain and stress
263     for ele_id in ele_axialforce.keys():
264         ele_axialforce[ele_id].append(ops.eleResponse(ele_id, 'force')[0]) # [N
265         ele_shearforce[ele_id].append(ops.eleResponse(ele_id, 'force')[1]) # [N
266         ele_momentforce[ele_id].append(ops.eleResponse(ele_id, 'force')[2]) # [N
267     # Store displacements
268     for node_id in node_displacementsX.keys():
269         node_displacementsX[node_id].append(ops.nodeDisp(node_id, 1))
270         node_displacementsY[node_id].append(ops.nodeDisp(node_id, 2))
271     else:
272         print('\n\nSTATIC ANALYSIS DONE.\n\n')
273
274     DATA = (reaction, disp_mid,
275             ele_axialforce, ele_shearforce, ele_momentforce,
276             node_displacementsX, node_displacementsY,
277             dispX, dispY)
278
279     return DATA
280
281 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
282     IDctrlDOF = 2 # 1: Horizontal Displcement - 2: Vertical Displcement
283     DINCR = -0.1 # [mm] Incremental Vertical Displacement
284     DMAX = -200.0 # [mm] Max. Vertical Displacement
```

Undeformed and Deformed Shapes - DEFORMED SCALE: 1.00

IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 699, Col 14 UTF-8 CRLF RW Mem 44%



PUSHOVER ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

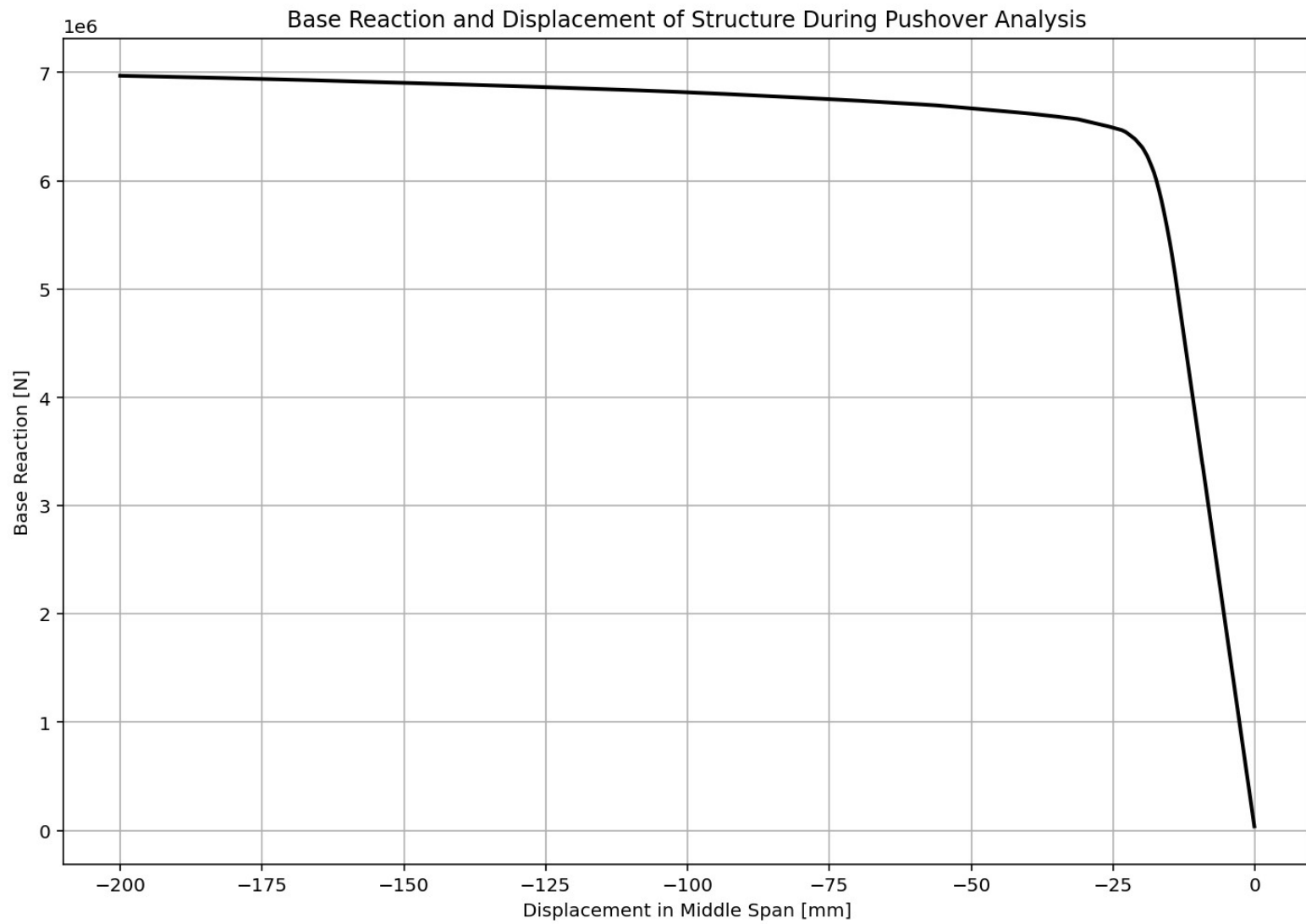
P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py x PEDESTRIAN_BRIDGE...EIGHT_FUN_EXTRA.py x

```
280
281 if ANAL_TYPE == 'PUSHOVER': # STATIC TIME-HISTORY ANALYSIS
282     IDctrlDOF = 2 # 1: Horizontal Displacement - 2: Vertical Displacement
283     DINCR = -0.1 # [mm] Incremental Vertical Displacement
284     DMAX = -200.0 # [mm] Max. Vertical Displacement
285     ops.integrator('DisplacementControl', center_node, IDctrlDOF, DINCR)
286     ops.analysis('Static')
287     Nsteps = int(np.abs(DMAX/ DINCR))
288     STEP = 0.0
289     for step in range(Nsteps):
290         OK = ops.analyze(1)
291         S02.ANALYSIS(OK, 1, MAX_TOLERANCE, MAX_ITERATIONS)
292         ops.reactions()
293         reaction.append(ops.nodeReaction(1, 2)+ops.nodeReaction(30, 2)) # SHEAR BASE R
294         disp_mid.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT
295         dispX.append(ops.nodeDisp(center_node, 1)) # DISPLACEMENT
296         dispY.append(ops.nodeDisp(center_node, 2)) # DISPLACEMENT
297         # Store elements force, strain and stress
298         for ele_id in ele_axialforce.keys():
299             ele_axialforce[ele_id].append(ops.eleResponse(ele_id, 'force')[0])
300             ele_shearforce[ele_id].append(ops.eleResponse(ele_id, 'force')[1])
301             ele_momentforce[ele_id].append(ops.eleResponse(ele_id, 'force')[2])
302         # Store displacements
303         for node_id in node_displacementsX.keys():
304             node_displacementsX[node_id].append(ops.nodeDisp(node_id, 1))
305             node_displacementsY[node_id].append(ops.nodeDisp(node_id, 2))
306         # IN EACH STEP, STRUCTURE PERIOD GOING TO BE CALCULATED
307         #PERIODmin, PERIODmax = S06.RAYLEIGH_DAMPING(3, 0.5*DR, DR, 0, 1)
308         PERIODmin, PERIODmax = S05.EIGENVALUE_ANALYSIS(3, PLOT=True)
309         PERIOD_MIN.append(PERIODmin)
310         PERIOD_MAX.append(PERIODmax)
311         # SECTION STRESS-STRAIN FOR BOTTOM FIBER
312         stressB, strainB = S10.SECTION_STRESS_STRAIN(center_node-1, SEC_TAG, -Depth*0.
313         # Store in separate lists
```

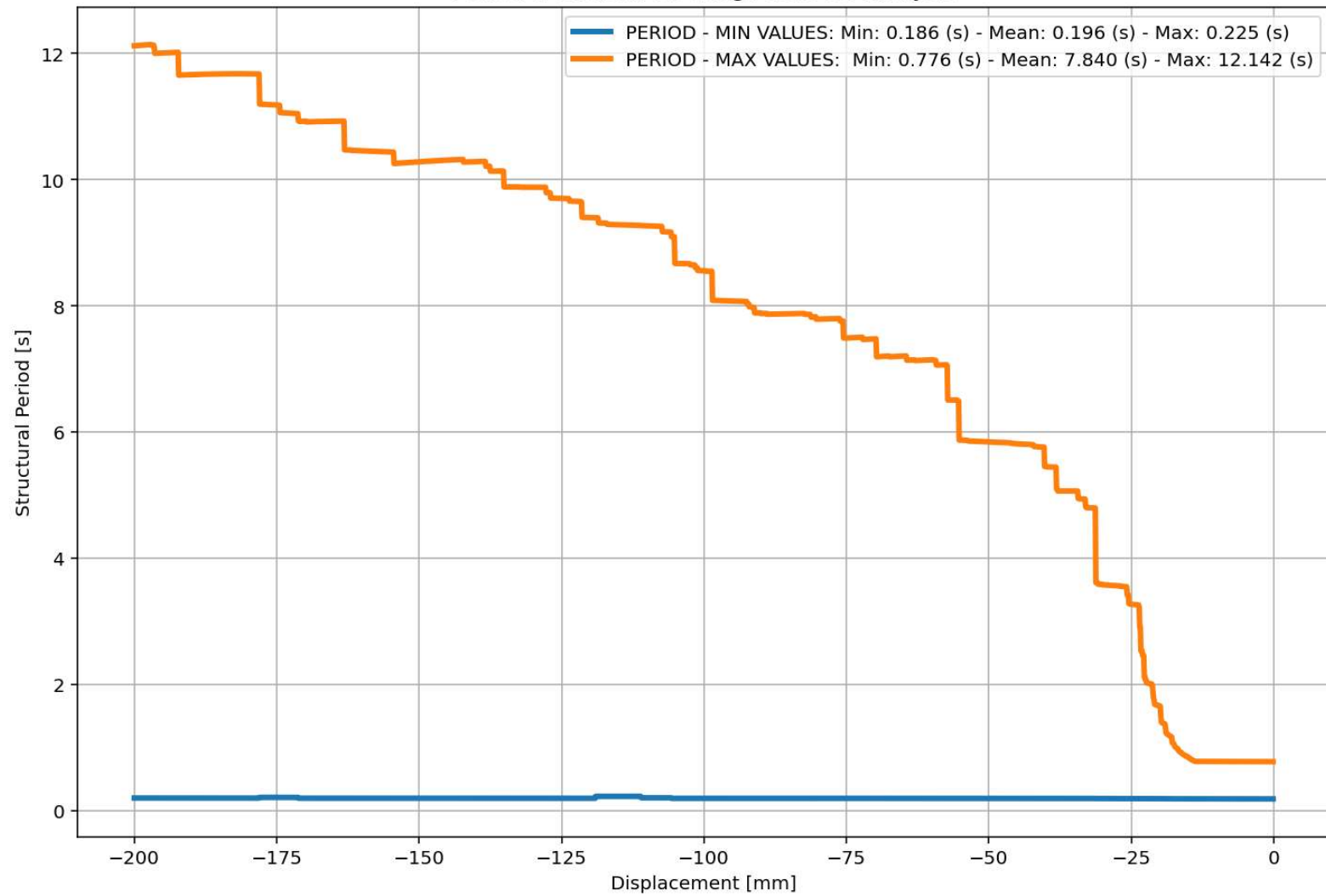
Base Reaction and Displacement of Structure During Pushover Analysis

IPython Console Files Help Variable Explorer Debugger Plots History

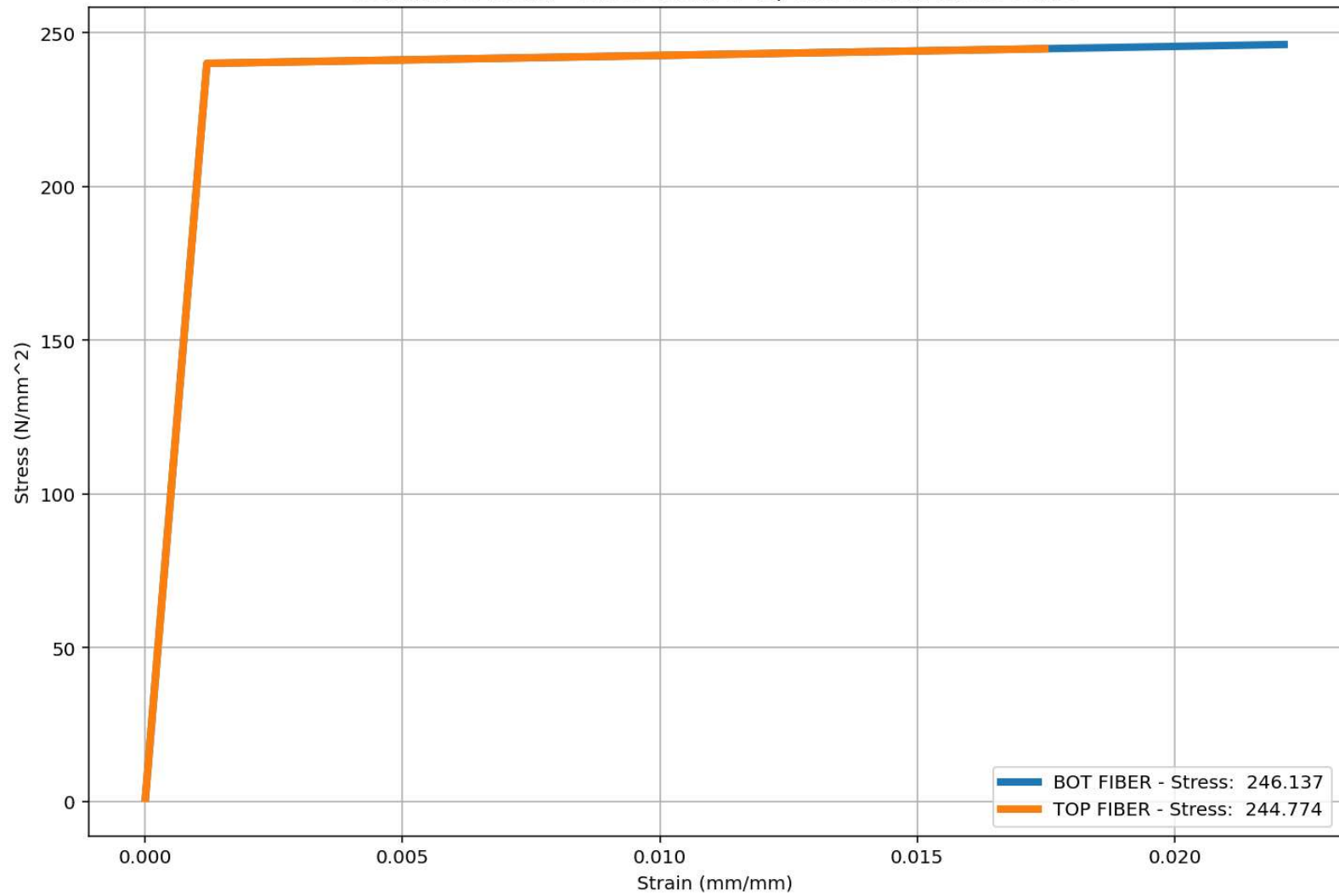
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 699, Col 14 UTF-8 CRLF RW Mem 45%

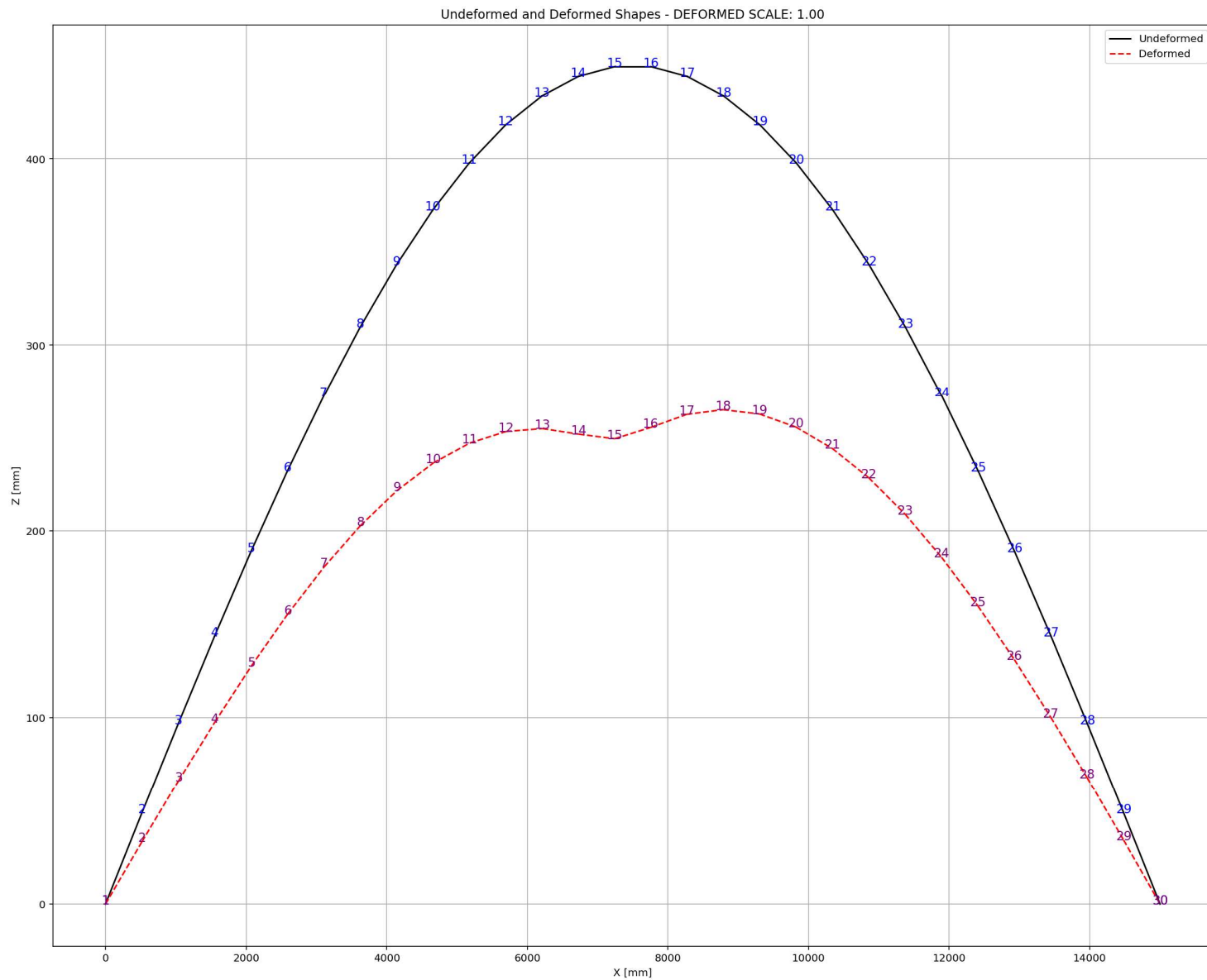


Period of Structure During Pushover Analysis



Behavior of beam - Stress-strain of Top and Bottom Fibers Curve





CYCLIC DISPLACEMENT ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py x PEDESTRIAN_BRIDGE...EIGHT_FUN_EXTRA.py x

```
337
338
339 if ANAL_TYPE == 'CYCLIC_DISPLACEMENT': # STATIC TIME-HISTORY ANALYSIS
340     IDctrlDOF = 2 # 1: Horizontal Displacement - 2: Vertical Displacement
341     DMAX = -200.0 # [mm] Max. Vertical Displacement
342     n_points = 10000
343     # 4. CYCLIC DISPLACEMENT PROTOCOL
344     # Key strain points (same logic as your protocol)
345     key_disp = np.array([
346         0.0,
347         0.1*DMAX, -0.1*DMAX,
348         0.5*DMAX, -0.5*DMAX,
349         0.8*DMAX, -0.8*DMAX,
350         DMAX, -DMAX,
351         0.1*DMAX, -0.1*DMAX,
352         0.5*DMAX, -0.5*DMAX,
353         0.8*DMAX, -0.8*DMAX,
354         DMAX, -DMAX,
355         0.0
356     ])
357
358     # Generate 10000-point displacement protocol
359     disp_protocol = np.interp(
360         np.linspace(0, len(key_disp) - 1, n_points),
361         np.arange(len(key_disp)),
362         key_disp
363     )
364
365     ops.analysis('Static')
366     STEP = 0.0
367     for target_disp in disp_protocol:
368         current_disp = ops.nodeDisp(center_node, 2) # DISPLACEMENT APPLIED IN MIDDLE N
369         dU = target_disp - current_disp
370         ops.integrator('DisplacementControl', center_node, IDctrlDOF, dU)
371         OK = ops.analyze(1)
```

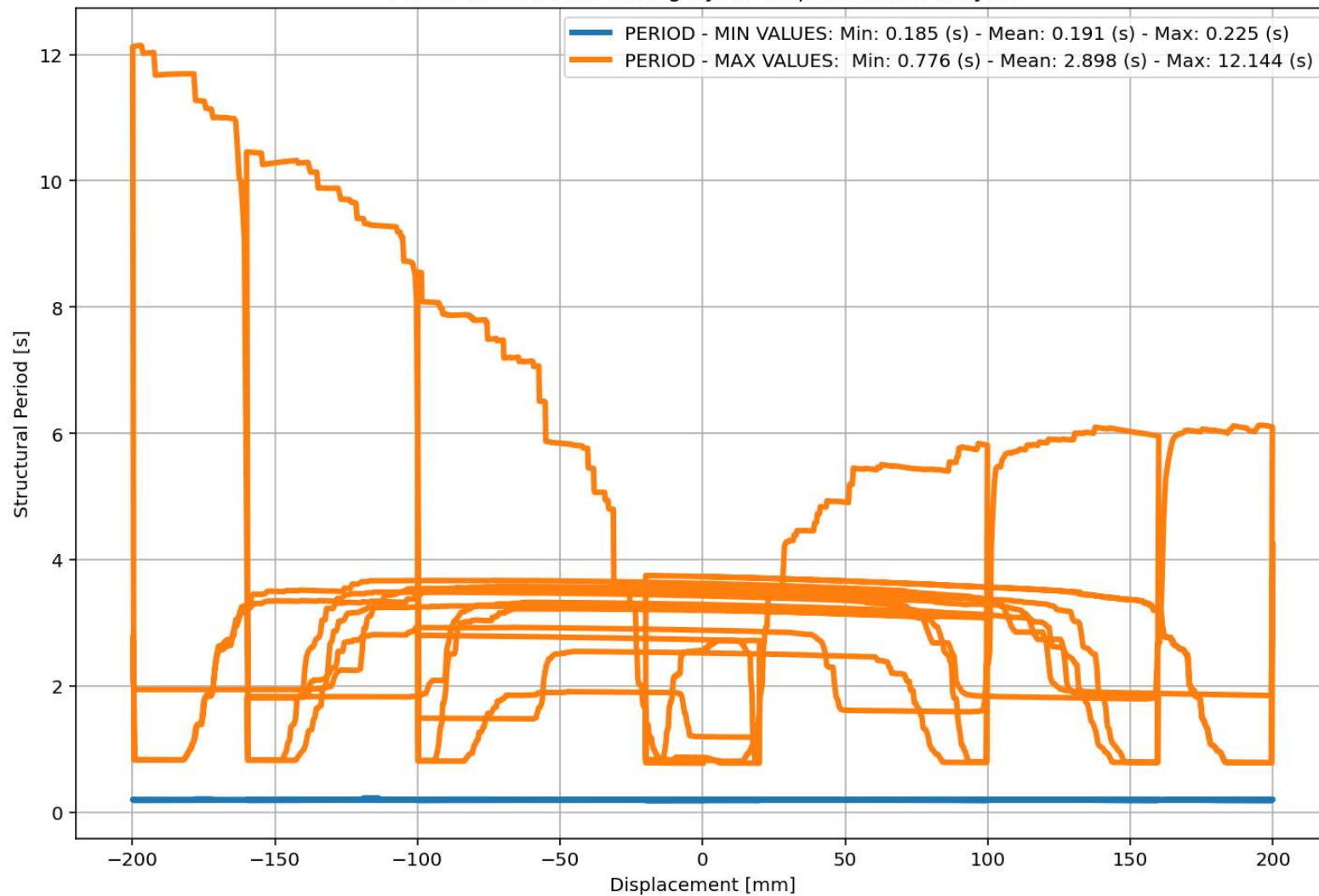
Base Reaction and Displacement of Structure During Cyclic-Displacement Analysis

IPython Console Files Help Variable Explorer Debugger Plots History

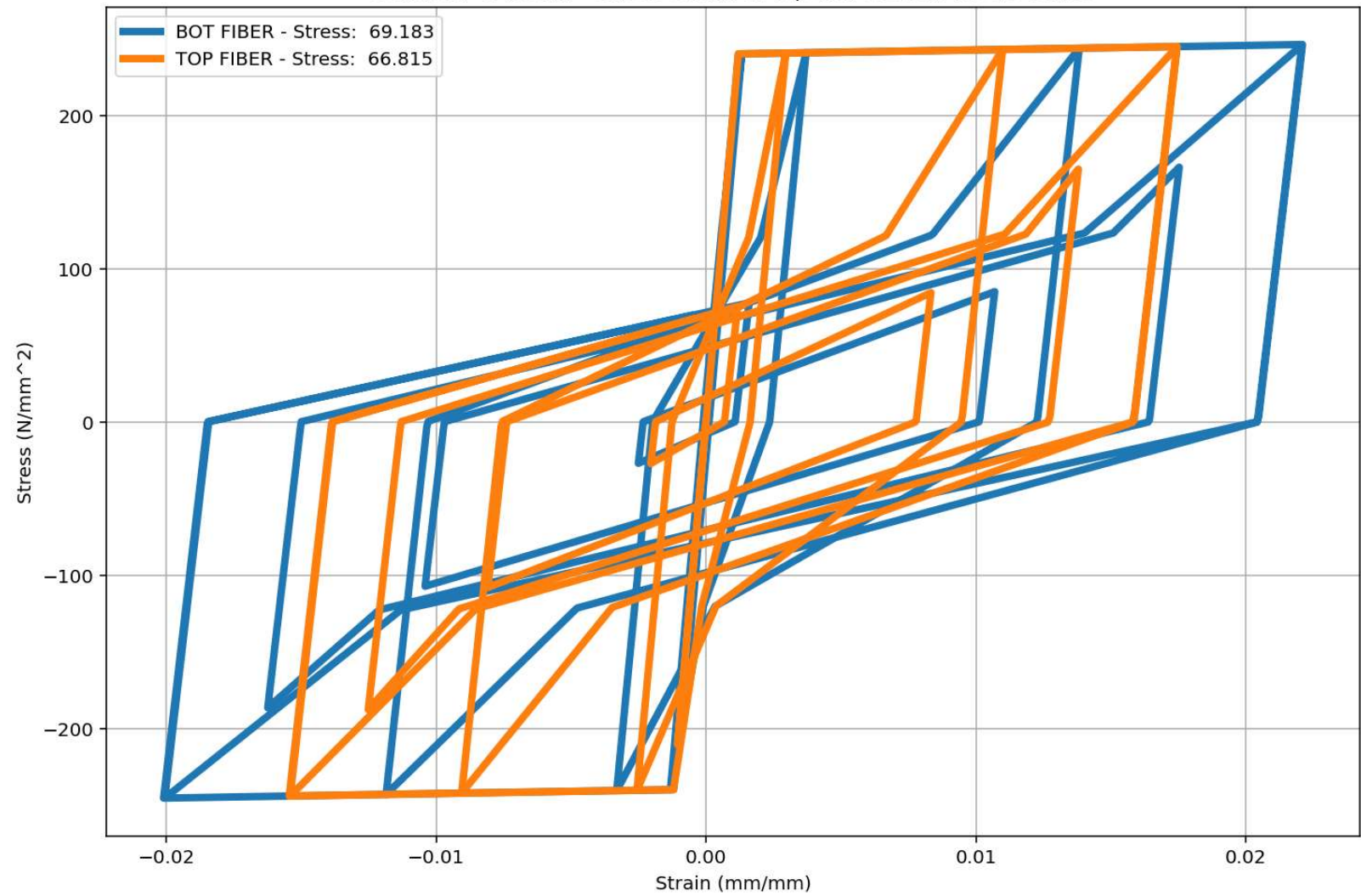
Inline Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 699, Col 14 UTF-8 CRLF RW Mem 46%

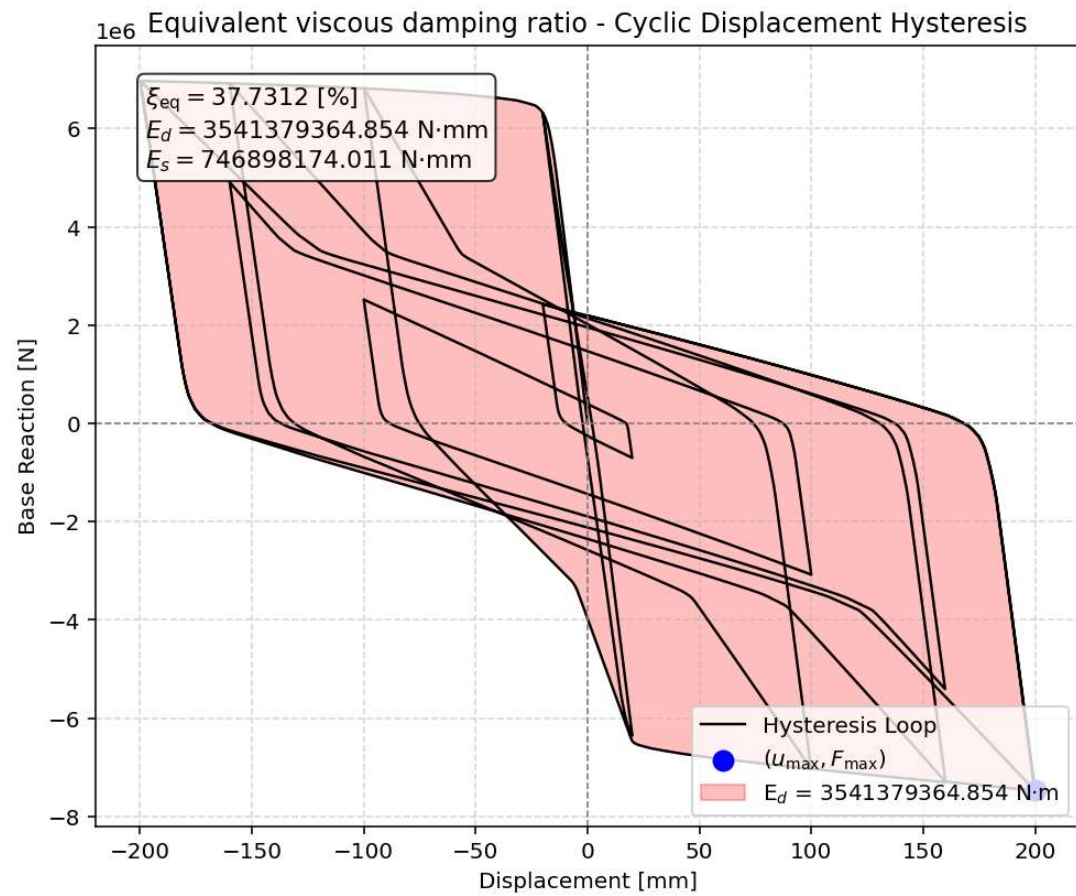


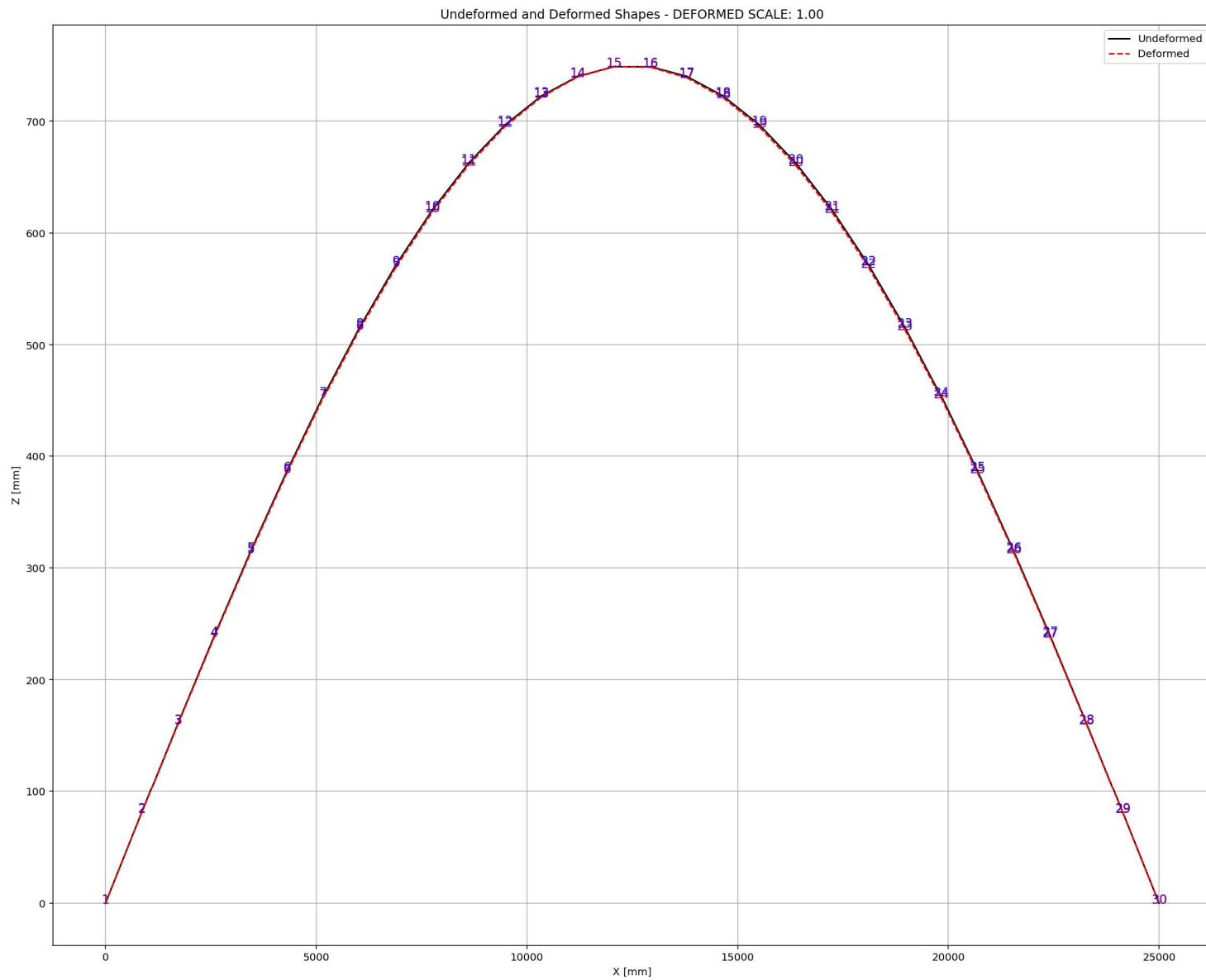
Period of Structure During Cyclic Displacement Analysis



Behavior of beam - Stress-strain of Top and Bottom Fibers Curve







STATIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py x PEDESTRIAN_BRIDGE...EIGHT_FUN_EXTRA.py x

```
417
418 if ANAL TYPE == 'STATIC EXTERNAL TIME-DEPENDENT LOADING': # STATIC TIME-HISTORY ANALYS
419     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
420     # IN HERE ANALYSIS TIME AND DURATION ARE LOAD STEPS
421     duration = 20.0 # [s] Analysis duration
422     dt = 0.01 # [s] Time step
423     DT = dt # [s] Time step
424     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
425     force_amplitude = 19600.0 # [N] Amplitude Force
426     omega_DT = 5.0715 # [rad/s] Natural angular frequency
427     time_steps = int(duration/dt)
428
429     # Check function
430     def CHECK_FUN(DT_time ,duration):
431         if DT_time > duration:
432             print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
433             exit()
434         return -1
435
436     CHECK_FUN(DT_time ,duration)
437     def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 s
438         import numpy as np
439         import matplotlib.pyplot as plt
440         # External Load Durations
441         num_steps = int(DT_time / DT)
442         load_time = np.linspace(0, DT_time, num_steps)
443         target_frequency = 1.0 * omega_DT # Target excitation frequency
444         DT_load = force_amplitude * np.sin(target_frequency * load_time)
445         # Plot External Time-dependent Loading
446         plt.figure(figsize=(10, 6))
447         plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_load)}')
448         plt.title('External Time-dependent Loading Over Time')
449         plt.xlabel('Time (s)')
450         plt.ylabel('Force (N)')
```

External Time-dependent Loading Over Time

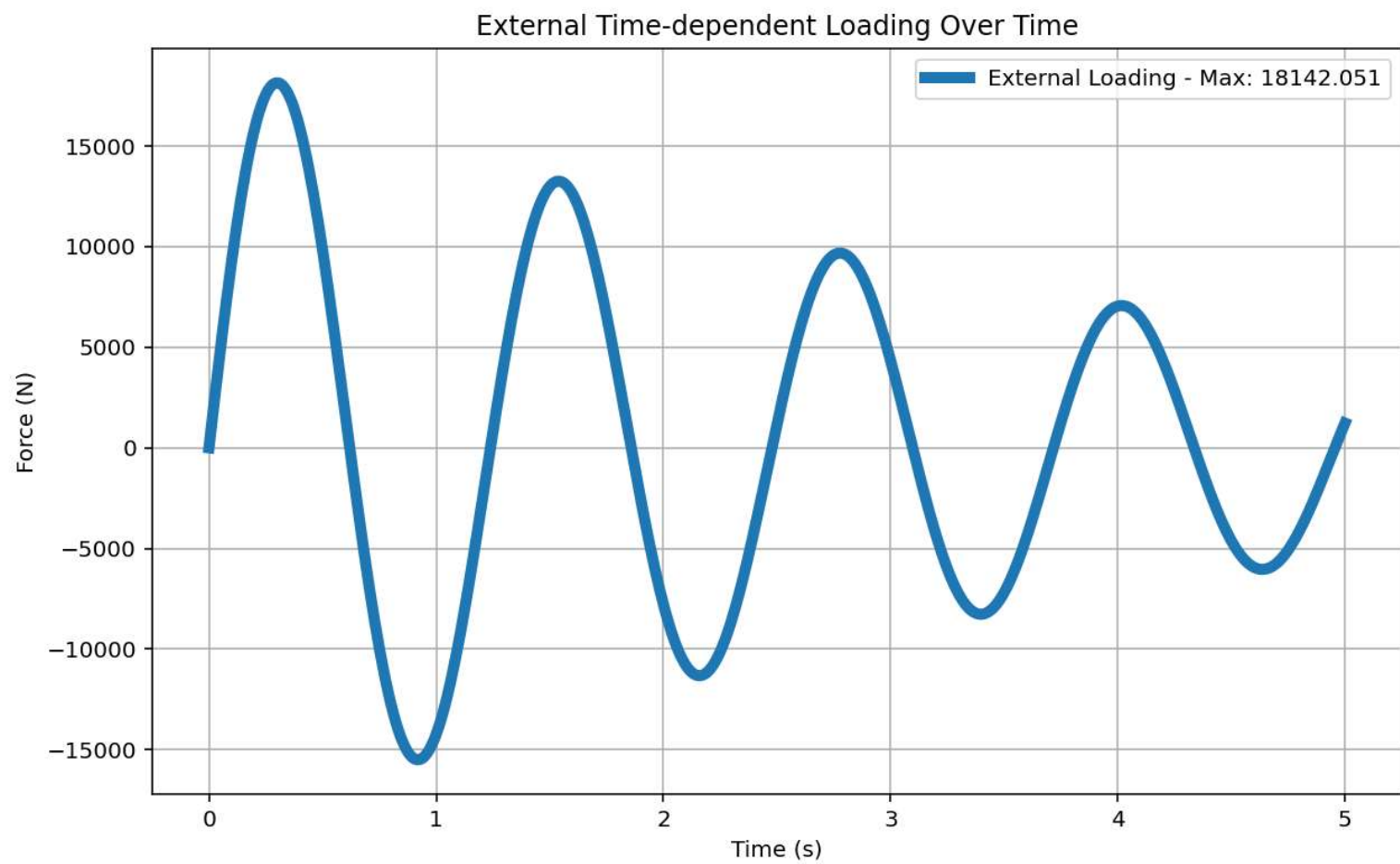
Force (N)

Time (s)

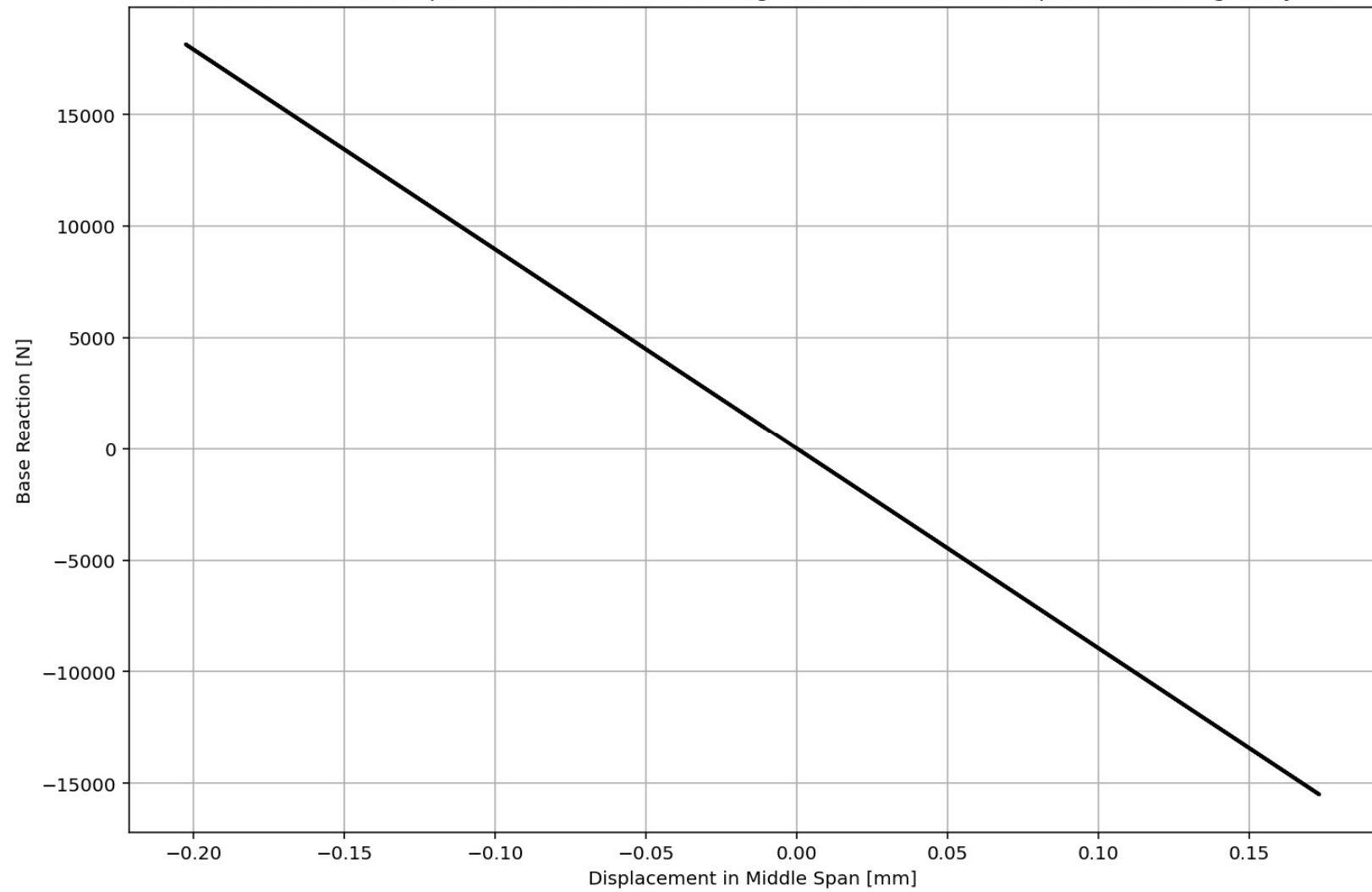
External Loading - Max: 18142.051

Python Console Files Help Variable Explorer Debugger Plots History

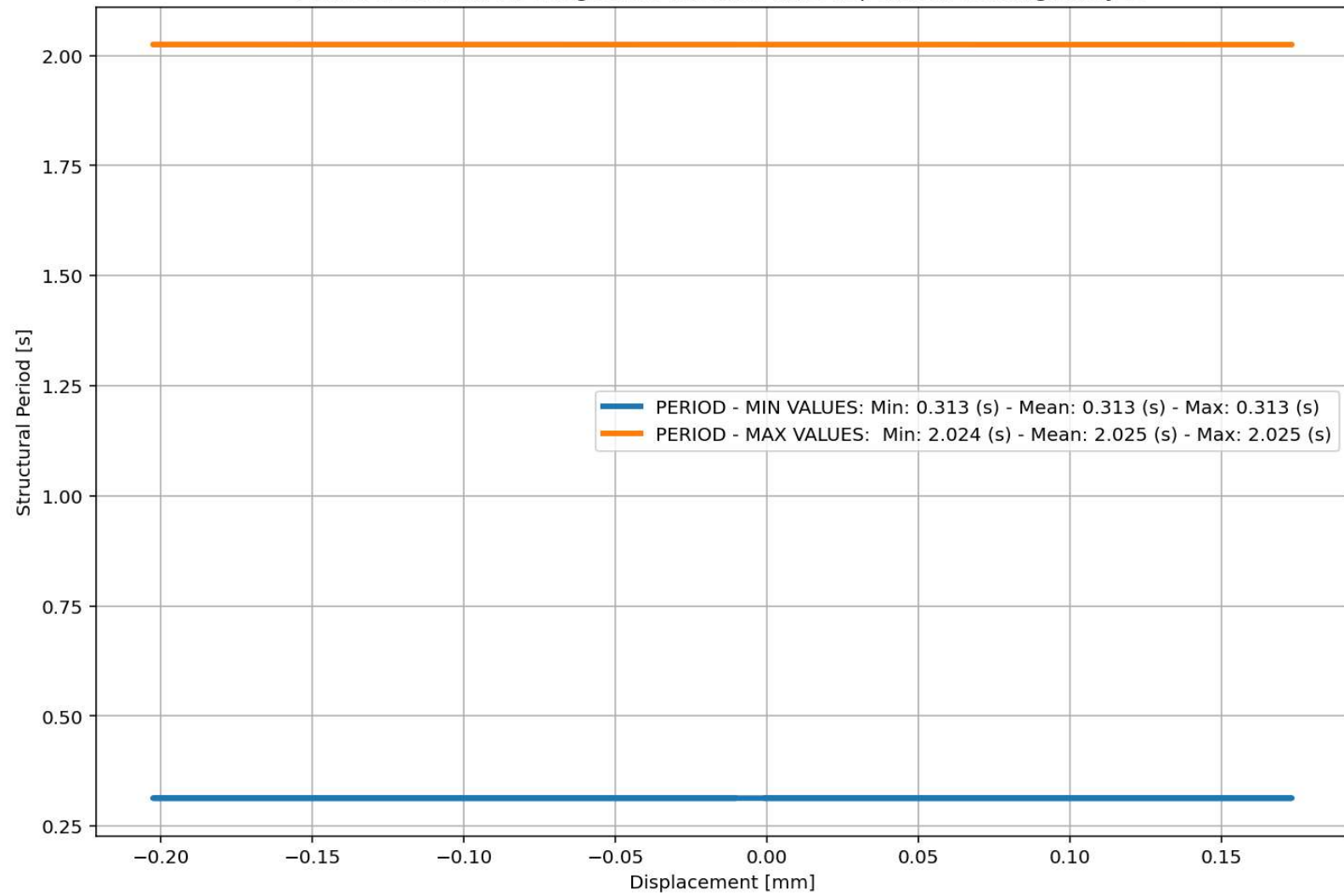
Inline Conda: anaconda3 (Python 3.12.7) LSP: Python Line 699, Col 14 UTF-8 CRLF RW Mem 46%



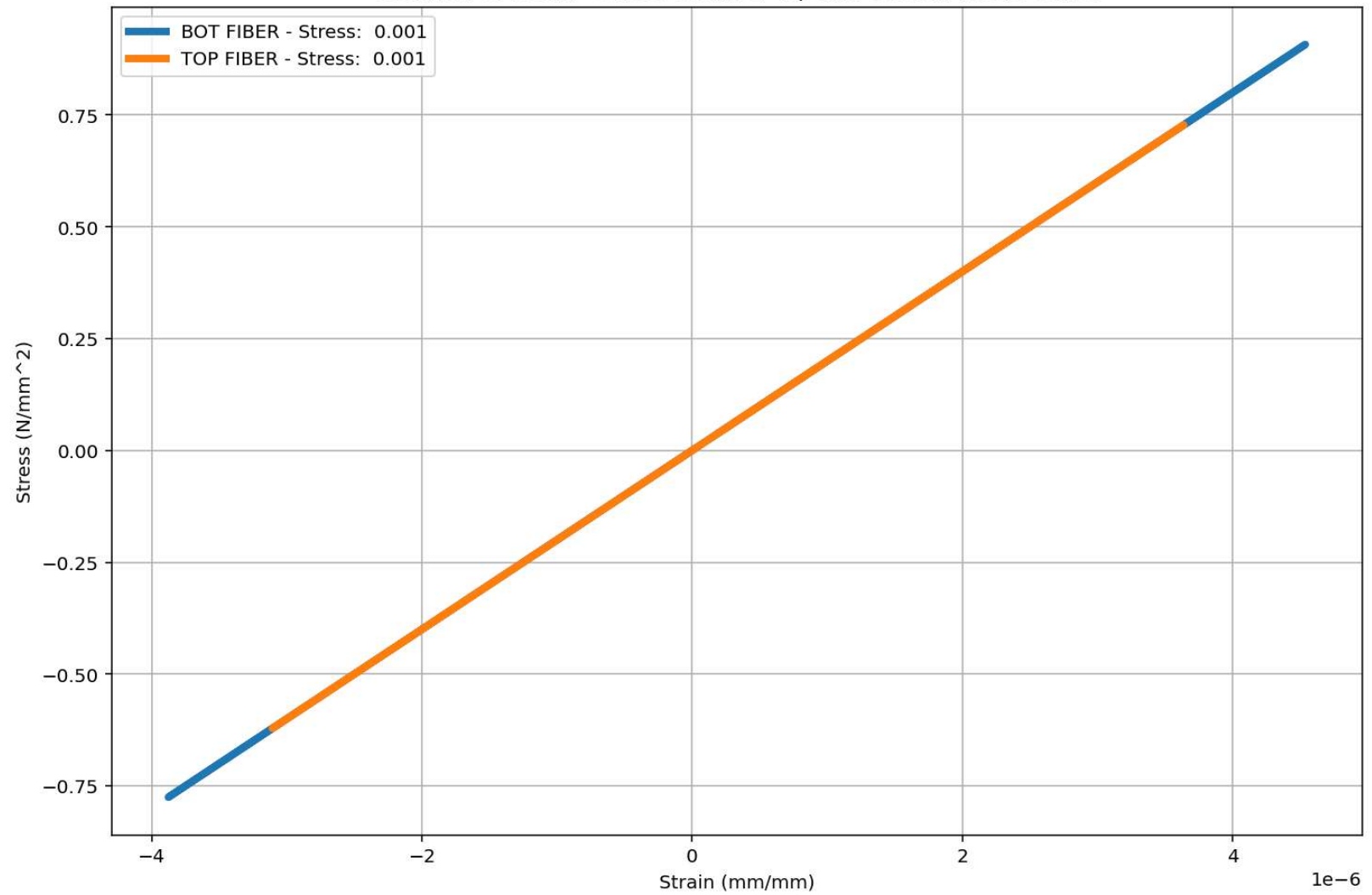
Base Reaction and Displacement of Structure During Static External Time-dependent Loading Analysis

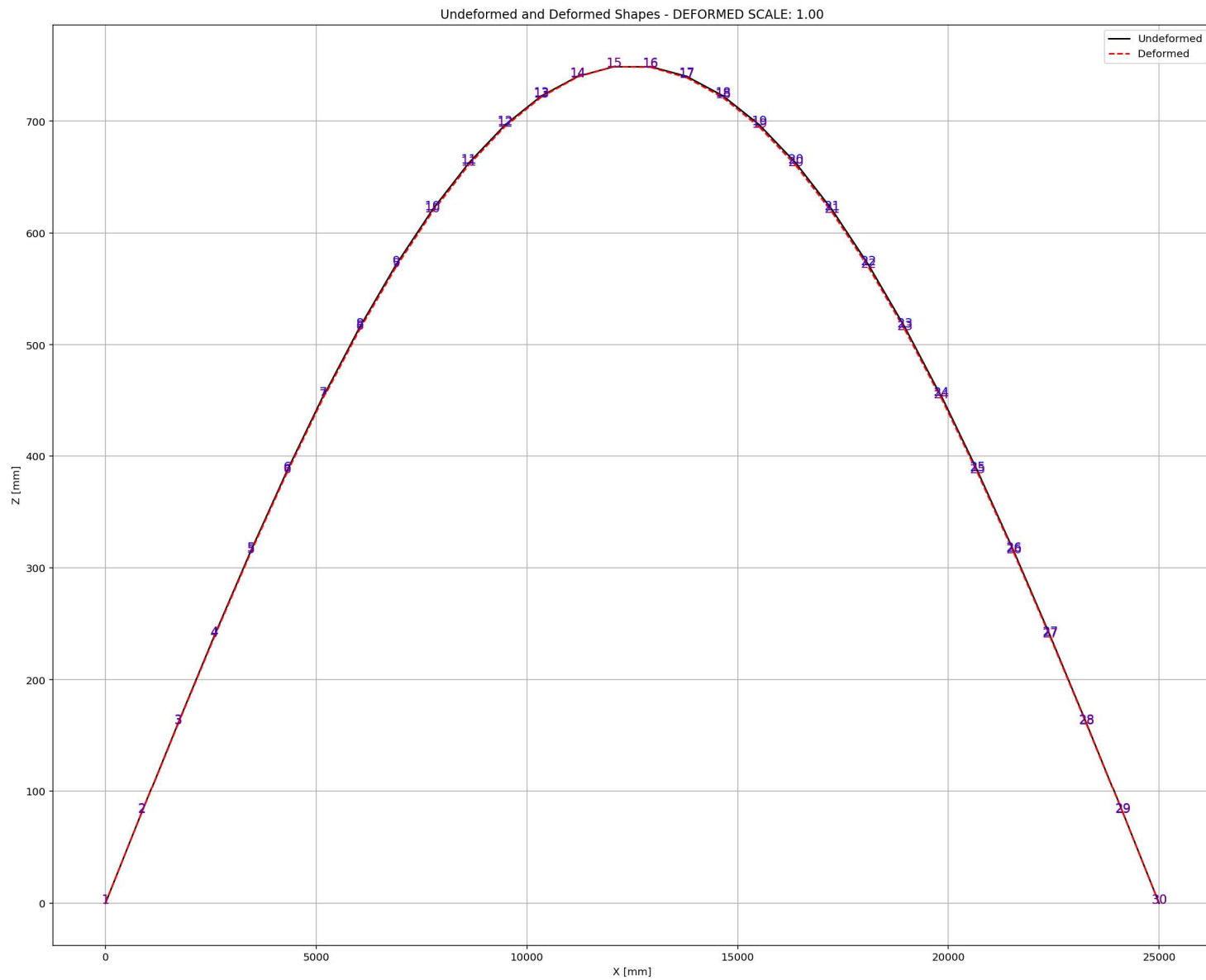


Period of Structure During Static External Time-dependent Loading Analysis



Behavior of beam - Stress-strain of Top and Bottom Fibers Curve





DYNAMIC TIME-HISTORY WITH EXTERNAL TIME-DEPENDENT LOADING ANALYSIS

Spyder (Python 3.12)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py x PEDESTRIAN_BRIDGE...EIGHT_FUN_EXTRA.py x

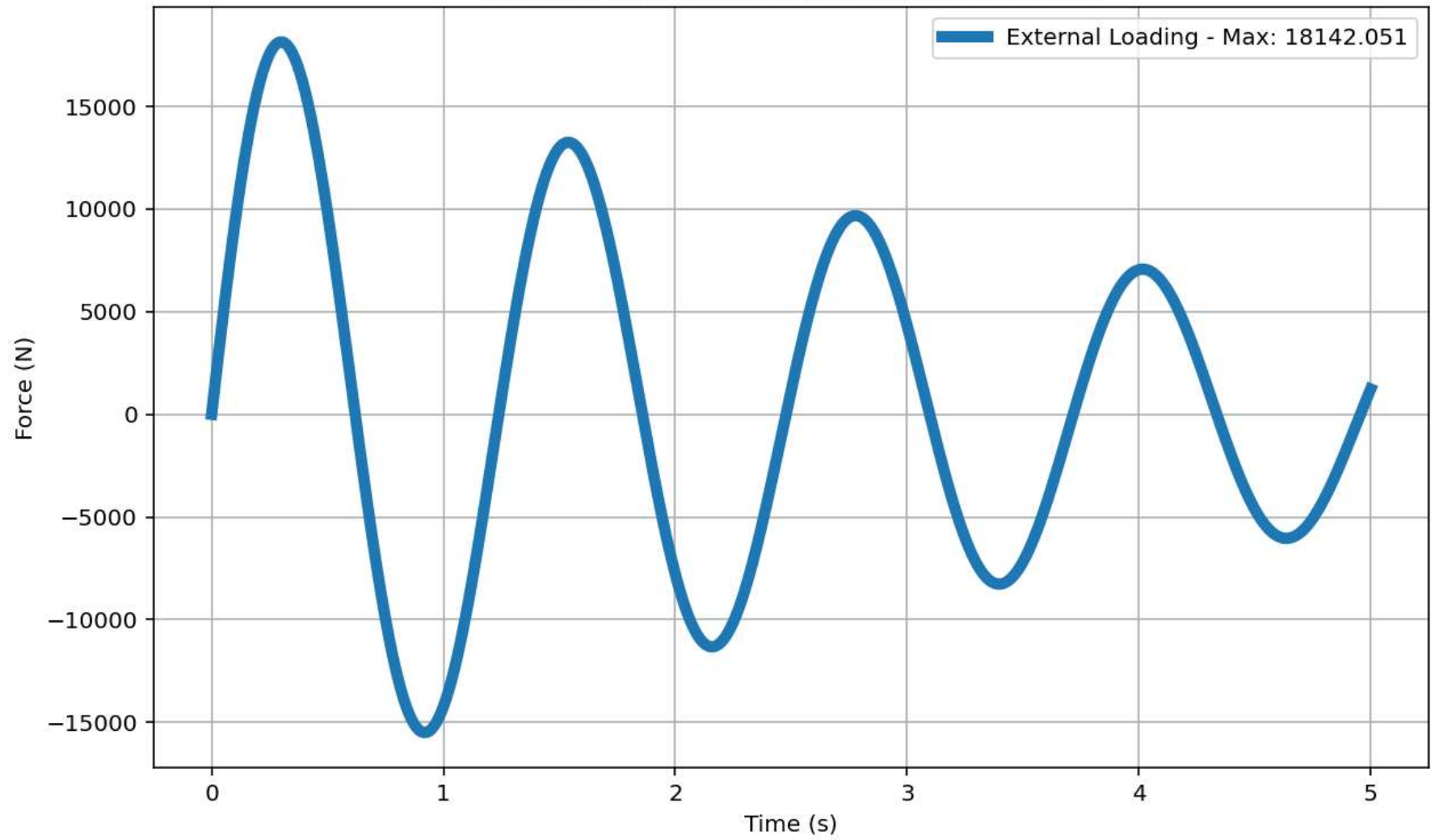
```
545
546 if ANAL_TYPE == 'DYNAMIC EXTERNAL TIME-DEPENDENT LOADING': # DYNAMIC TIME-HISTORY ANAL
547     %% DEFINE EXTERNAL TIME-DEPENDENT LOADING PROPERTIES
548     duration = 20.0 # [s] Analysis duration
549     dt = 0.001 # [s] Time step
550     DT = dt # [s] Time step
551     DT_time = 5.0 # [s] Total external Load Analysis Durations [*****]
552     force_amplitude = 19600.0 # [N] Amplitude Force
553     omega_DT = 5.0715 # [rad/s] Natural angular frequency
554     DR = 0.03 # Damping Ratio
555
556 # Check function
557 def CHECK_FUN(DT_time ,duration):
558     if DT_time > duration:
559         print('\n\nAnalysis Duration Must be greater than External Load Duration\n')
560         exit()
561     return -1
562
563 CHECK_FUN(DT_time ,duration)
564 def EXTERNAL_TIME_DEPENDENT(force_amplitude, omega_DT, DT, DT_time): # P(t) = P0 s
565     import numpy as np
566     import matplotlib.pyplot as plt
567     # External Load Durations
568     num_steps = int(DT_time / DT)
569     load_time = np.linspace(0, DT_time, num_steps)
570     target_frequency = 1.0 * omega_DT # Target excitation frequency
571     DT_load = force_amplitude * np.sin(target_frequency * load_time)
572     # Plot External Time-dependent Loading
573     plt.figure(figsize=(10, 6))
574     plt.plot(load_time, DT_load, label=f'External Loading - Max: {np.max(DT_Load)}:
575     plt.title('External Time-dependent Loading Over Time')
576     plt.xlabel('Time (s)')
577     plt.ylabel('Force (N)')
578     plt.grid(True)
```

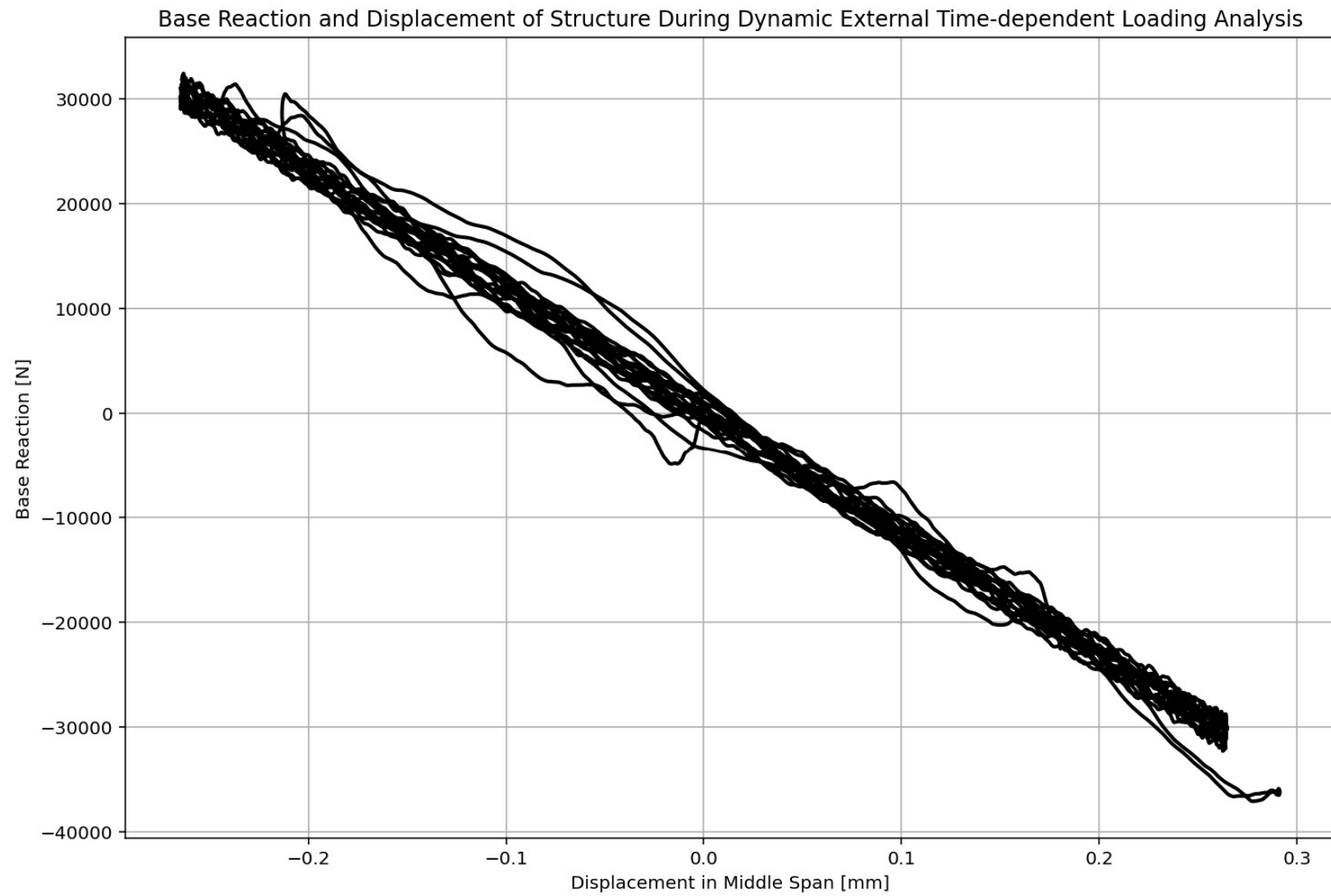
Base Reaction and Displacement of Structure During Dynamic External Time-dependent Loading Analysis

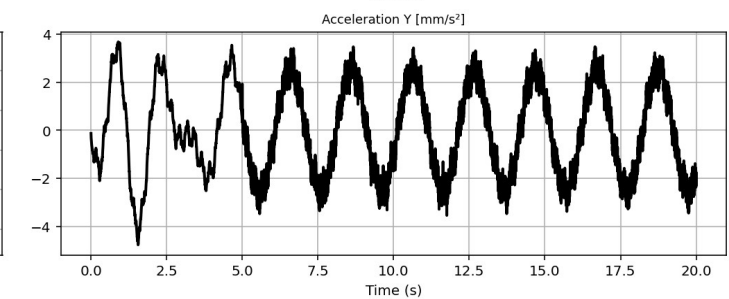
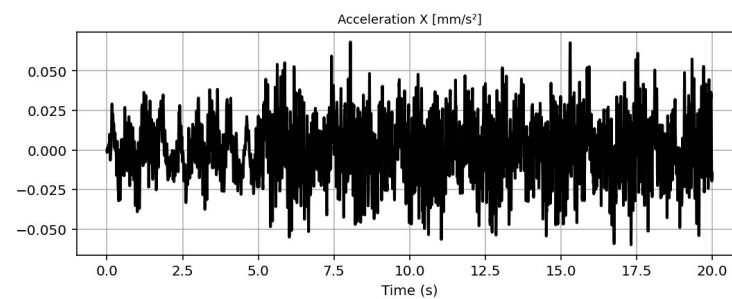
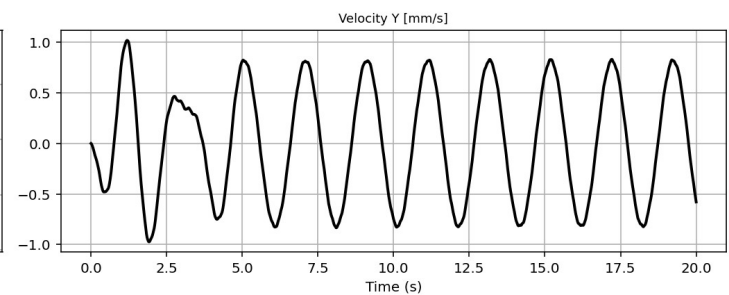
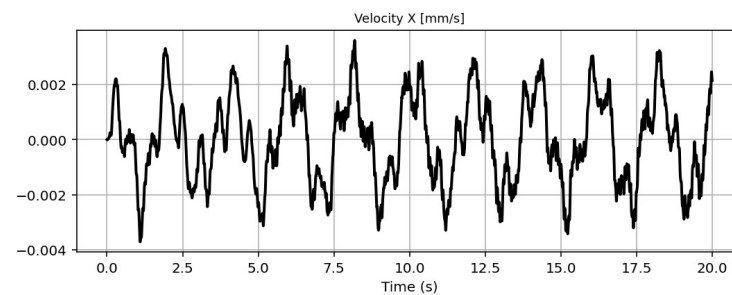
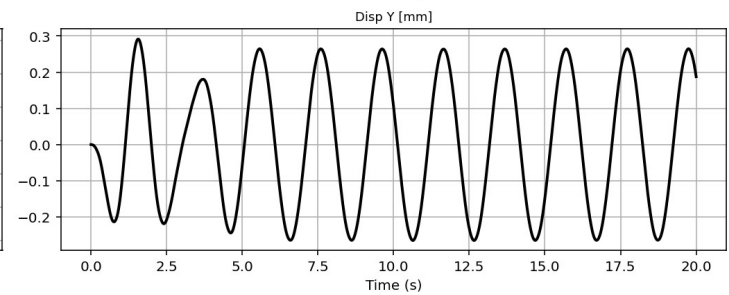
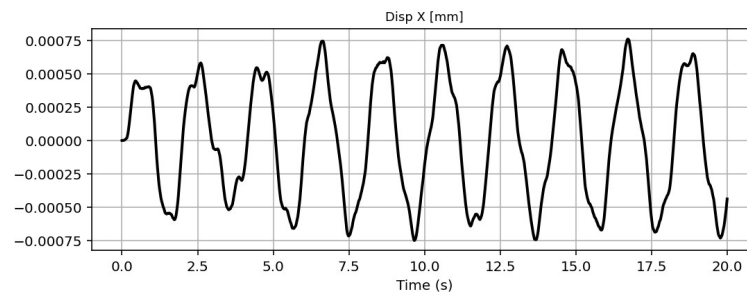
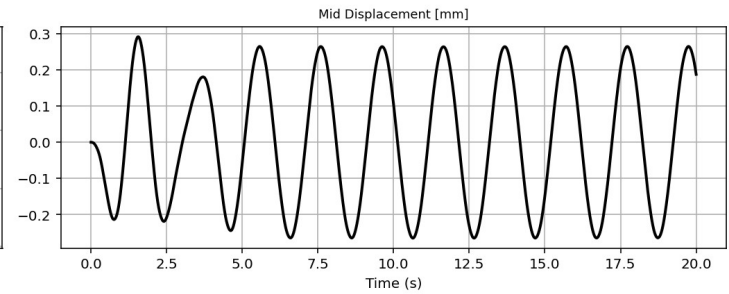
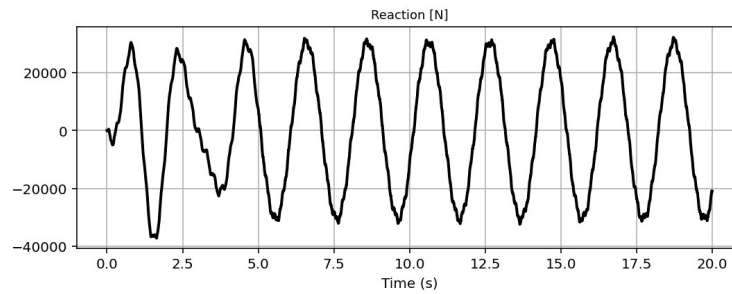
IPython Console Files Help Variable Explorer Debugger Plots History

Inline Conda: anaconda3 (Python 3.12.7) ✓ JSP: Python Line 699, Col 14 UTF-8 CRLF RW Mem 49%

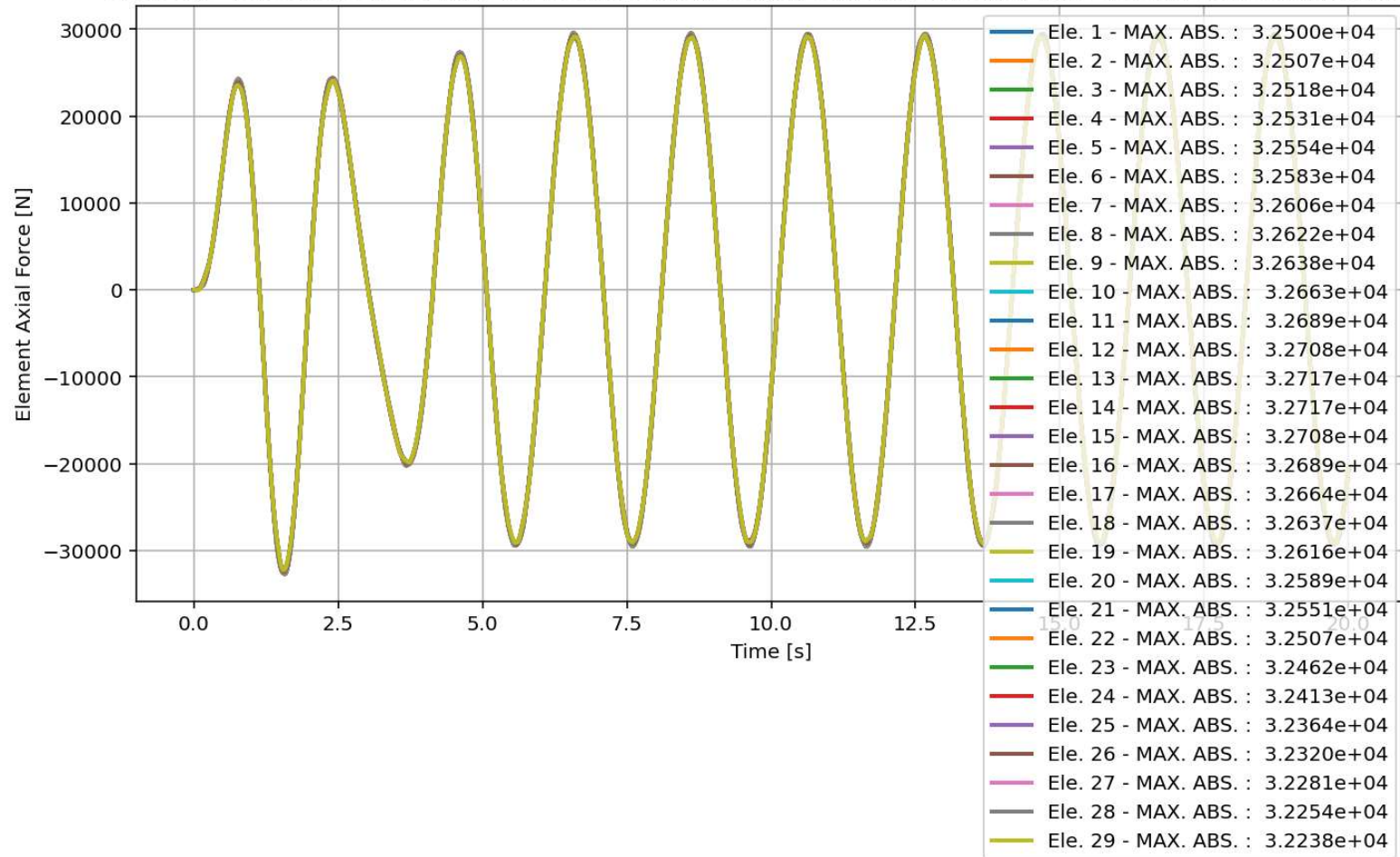
External Time-dependent Loading Over Time



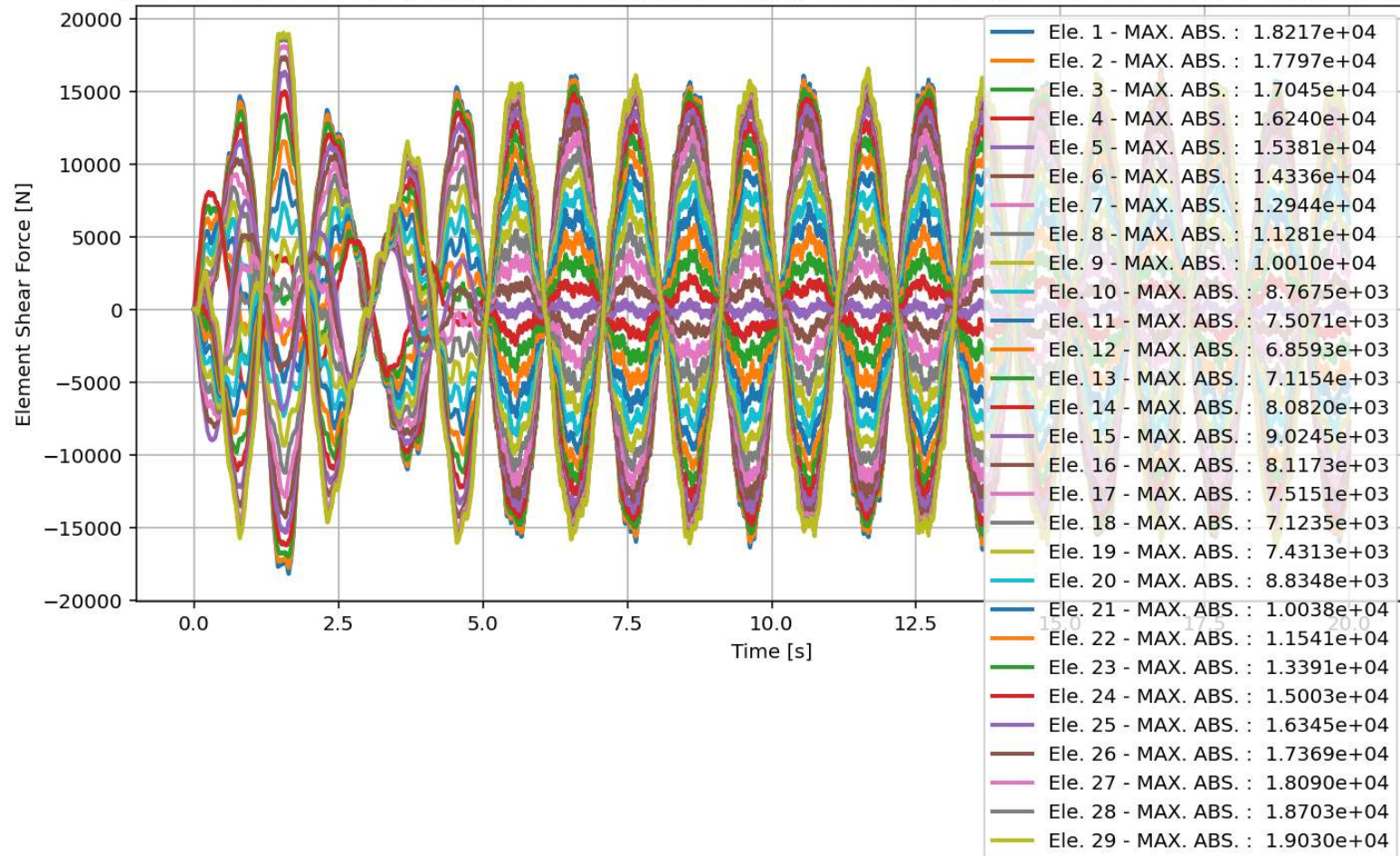




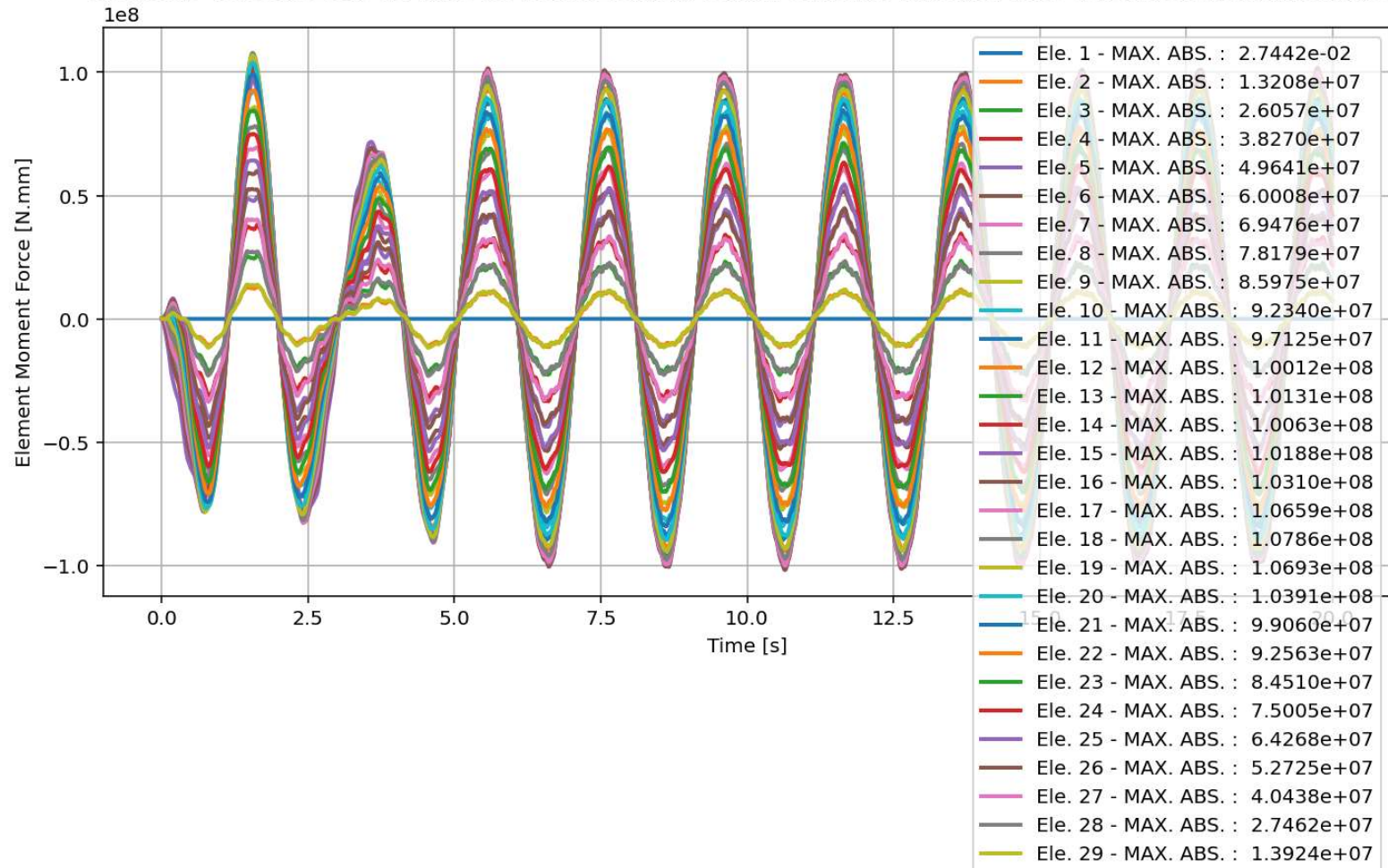
elements force in X Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



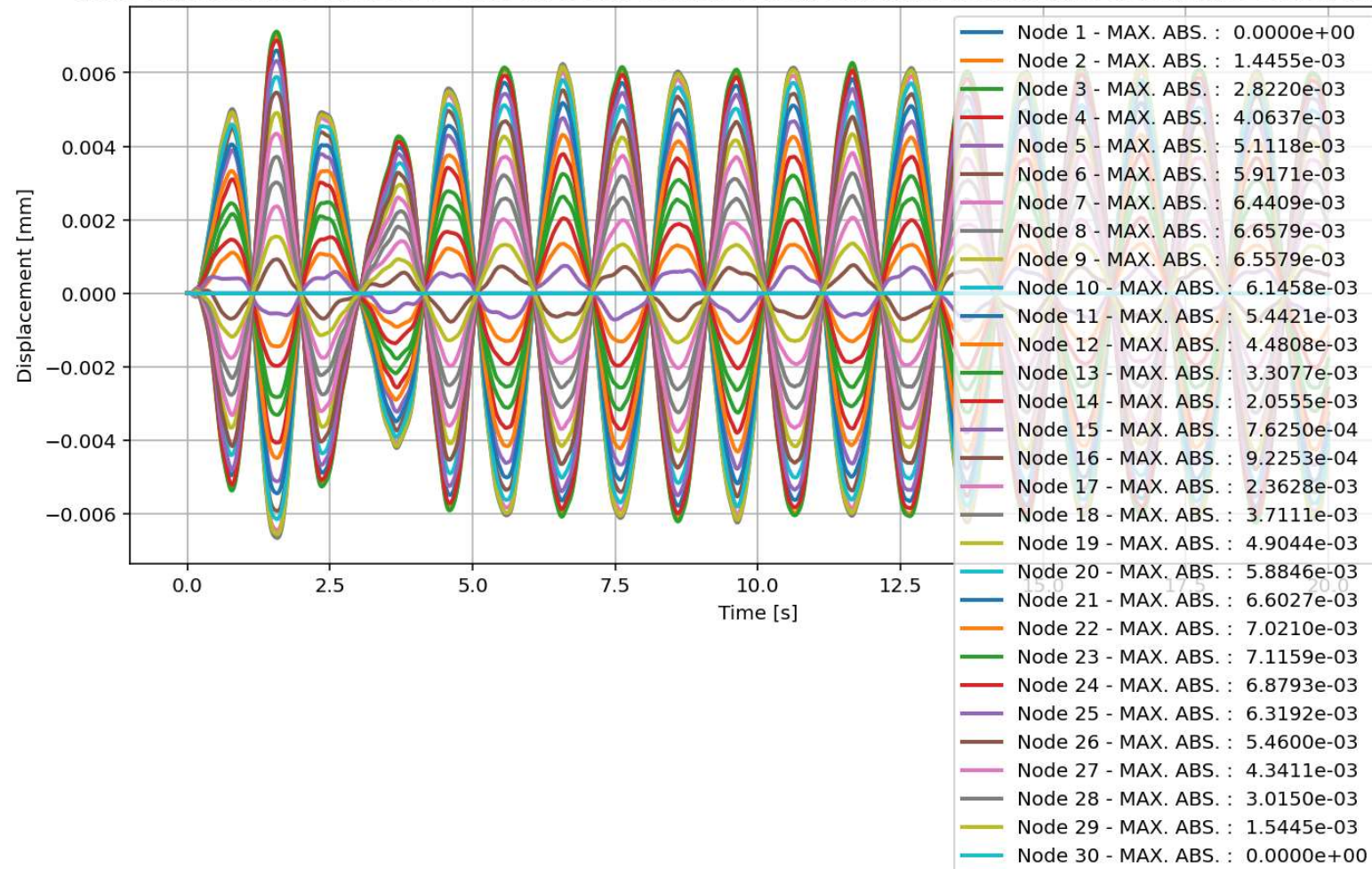
elements force in Y Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



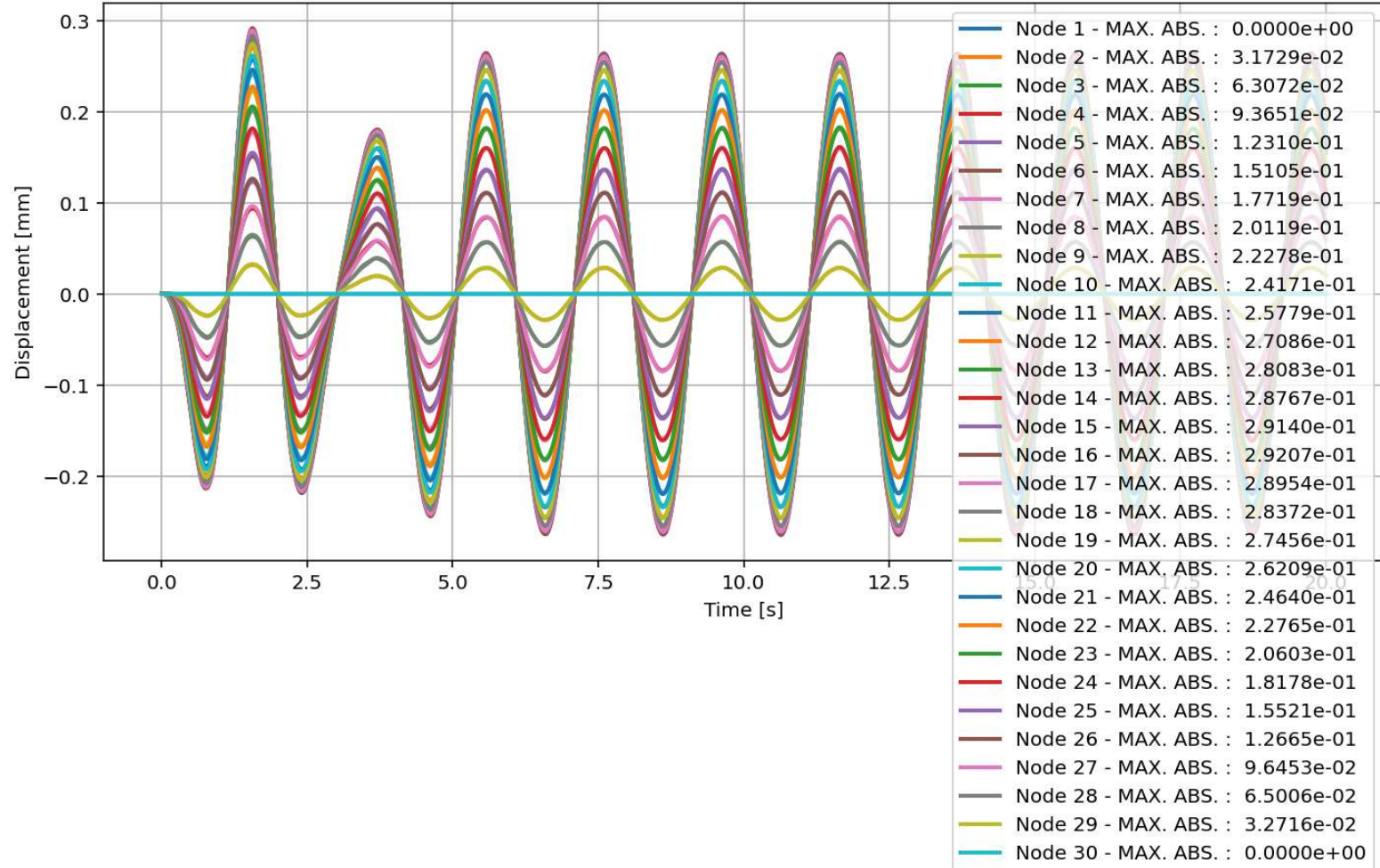
elements force in Z Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



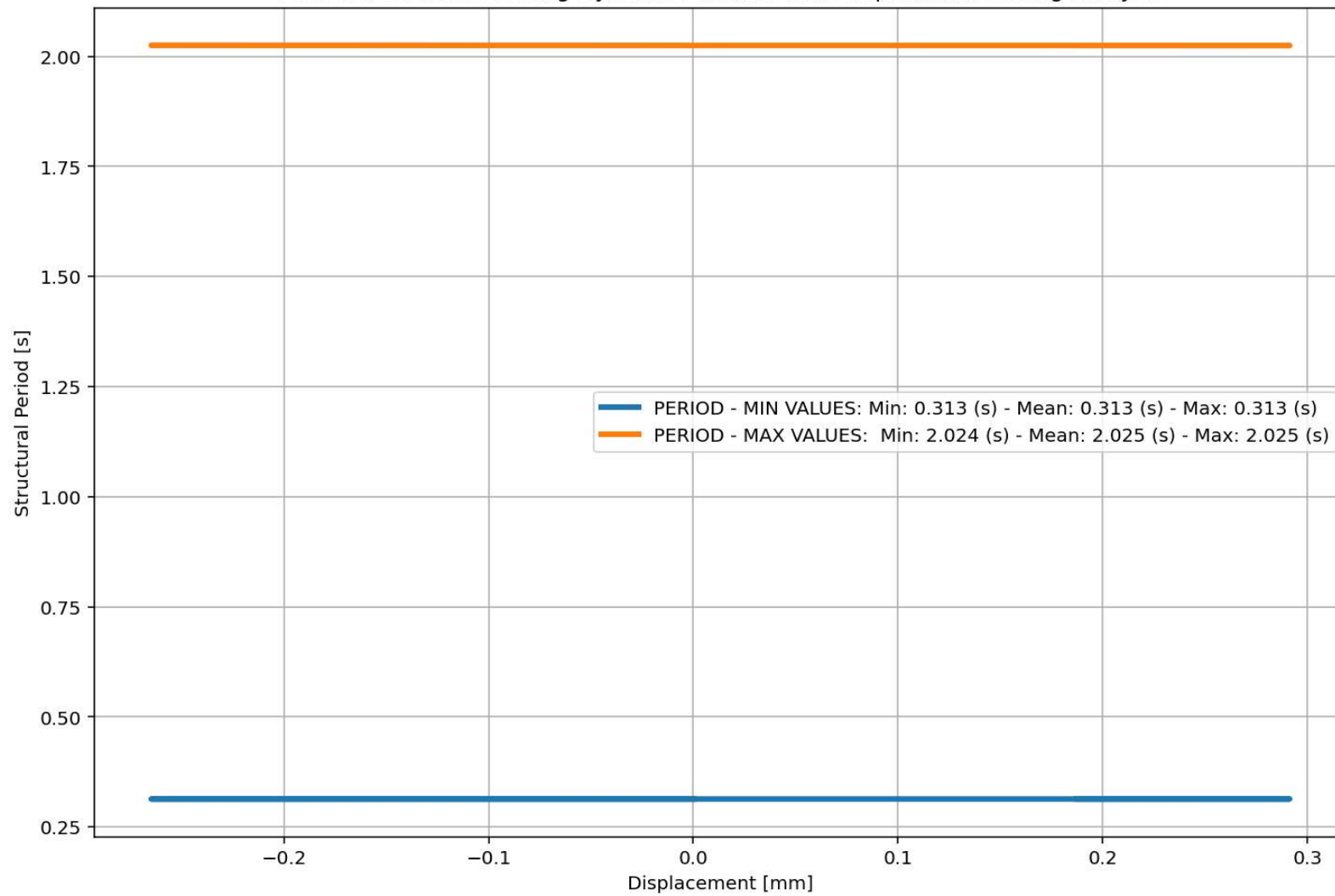
Node Displacements in X Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis



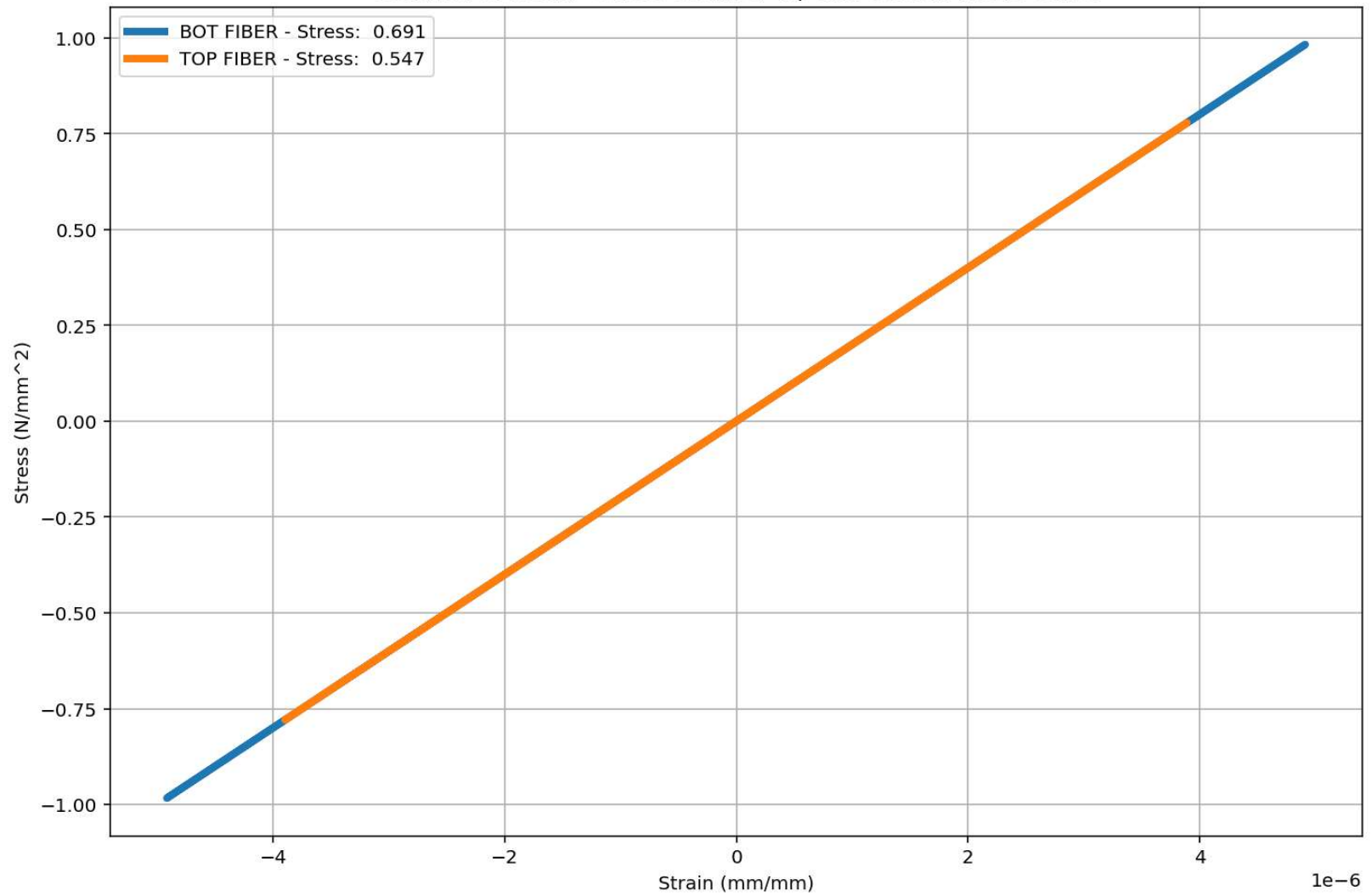
Node Displacements in Y Dir. vs Time for Beam Element During Dynamic External Time-dependent Loading Analysis

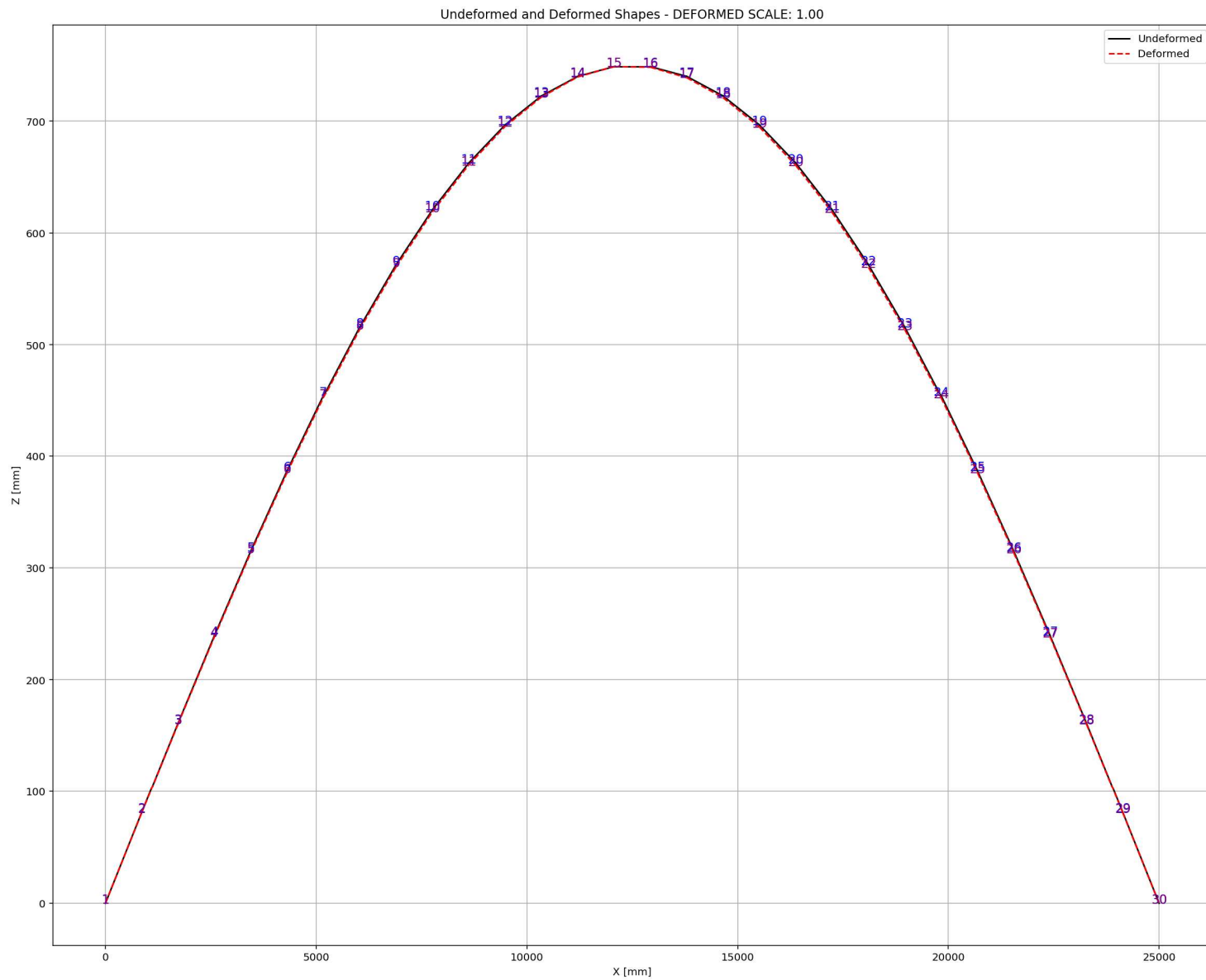


Period of Structure During Dynamic External Time-dependent Loading Analysis



Behavior of beam - Stress-strain of Top and Bottom Fibers Curve





FREE-VIBRATION ANALYSIS

Spyder (Python 3.12)

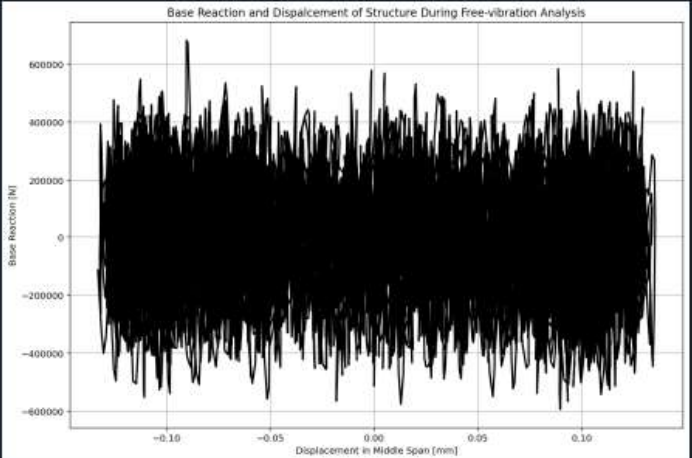
File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\Dell\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

P(t)_PEDESTRIAN_BR...I_SECTION_8_ANA.py X PEDESTRIAN_BRIDGE...EIGHT_FUN_EXTRA.py X

```
693
694 if ANAL_TYPE == 'FREE-VIBRATION': # DYNAMIC TIME-HISTORY ANALYSIS
695     %% DEFINE PARAMETERS FOR FREE-VIBRATION ANALYSIS
696     u0 = -0.00015 # [mm] Initial displacement
697     v0 = 5.15 # [mm/s] Initial velocity
698     a0 = 0.065 # [mm/s^2] Initial acceleration
699     IU = True # Free Vibration with Initial Displacement
700     IV = True # Free Vibration with Initial Velocity
701     IA = True # Free Vibration with Initial Acceleration
702     duration = 20.0 # [s] Analysis duration
703     dt = 0.001 # [s] Time step
704     DR = 0.03 # Damping Ratio
705
706     if IU == True:
707         # Define initial displacement
708         ops.setNodeDisp(center_node, 2, u0, '-commit') # INFO LINK: https://openseespy
709     if IV == True:
710         # Define initial velocity
711         ops.setNodeVel(center_node, 2, v0, '-commit') # INFO LINK: https://openseespy
712     if IA == True:
713         # Define initial acceleration
714         ops.setNodeAccel(center_node, 2, a0, '-commit') # INFO LINK: https://openseespy
715
716     ops.constraints('Plain')
717     ops.numberer('Plain')
718     ops.system('BandGeneral')
719     ops.test('NormDispIncr', MAX_TOLERANCE, MAX_ITERATIONS) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/
720     #ops.integrator('CentralDifference') # JUST FOR LINEAR ANALYSIS - INFO LINK: http://openseespydoc.readthedocs.io/en/latest/
721     alpha=0.5; beta=0.25;
722     ops.integrator('Newmark', alpha, beta) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/
723     #alpha=2/3; gamma=1.5-alpha; gamma=1.5-alpha; beta=(2-alpha)**2/4;
724     #ops.integrator('HHT', alpha, gamma, beta) # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/
725     ops.algorithm('Newton') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/
726     ops.analysis('Transient') # INFO LINK: https://openseespydoc.readthedocs.io/en/latest/
```

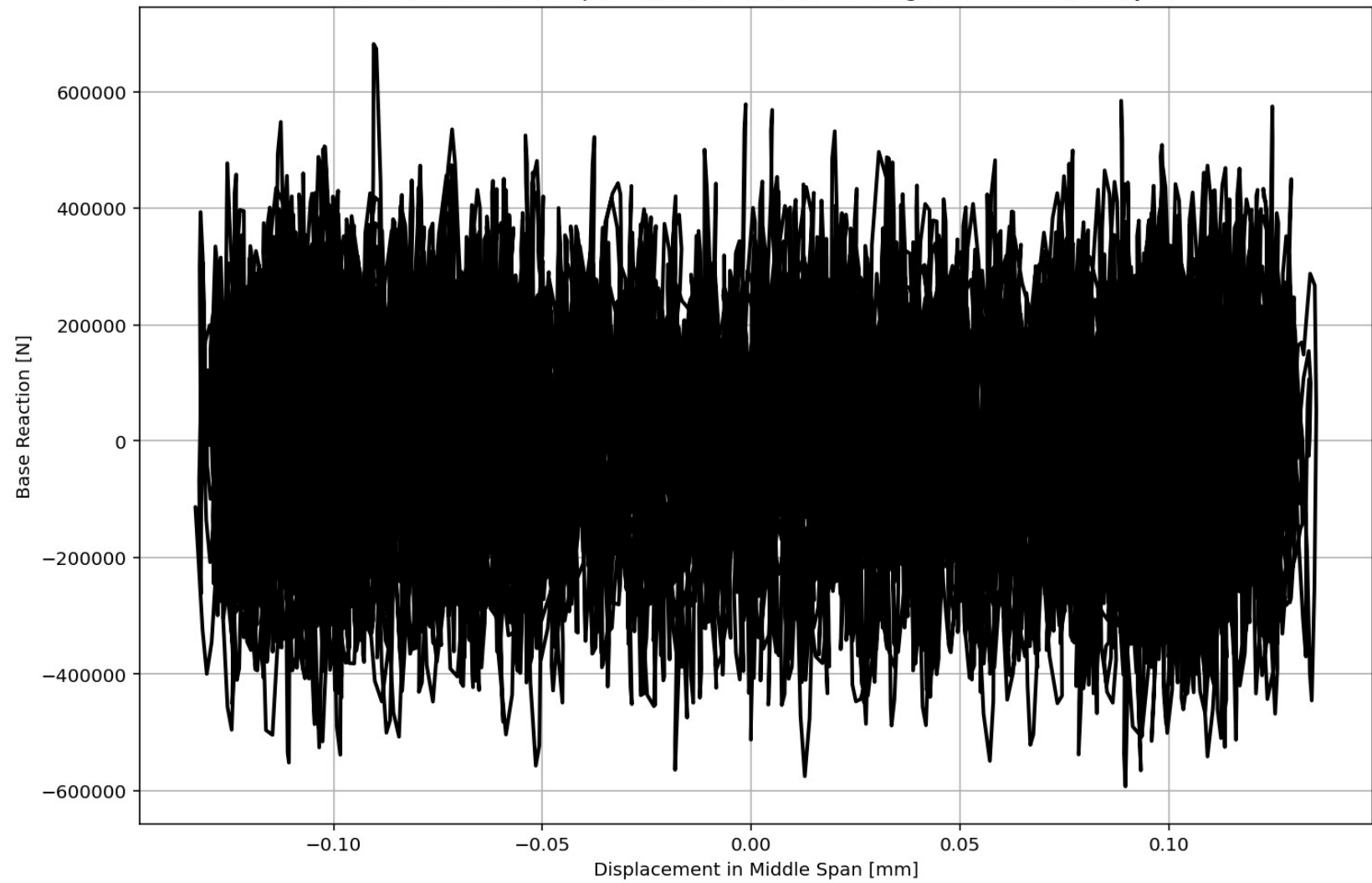
Base Reaction and Displacement of Structure During Free-vibration Analysis

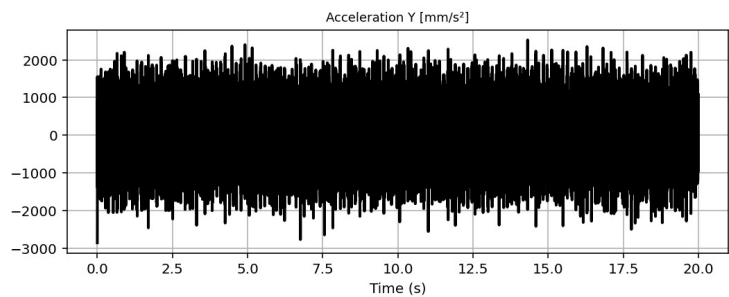
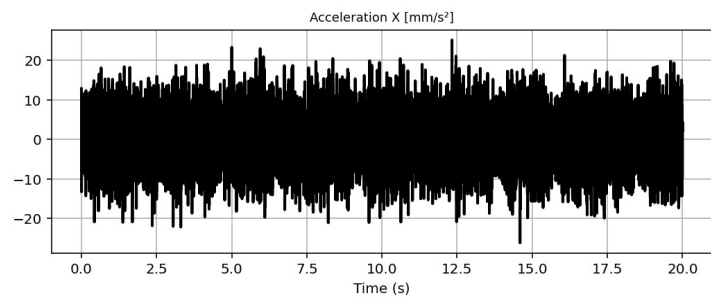
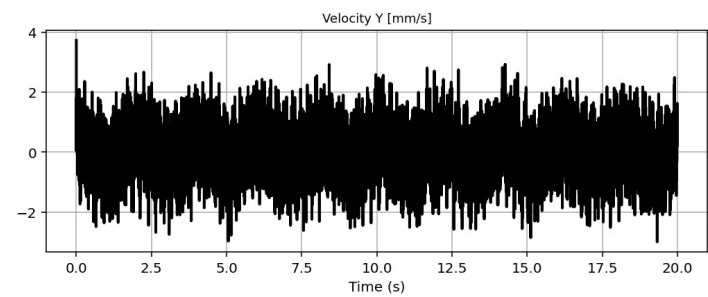
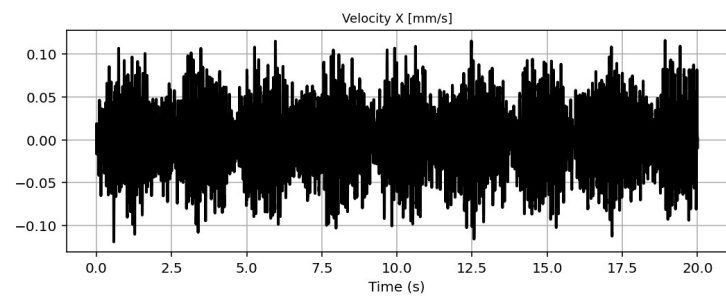
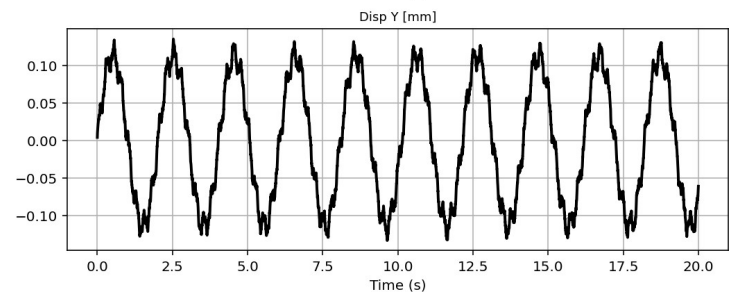
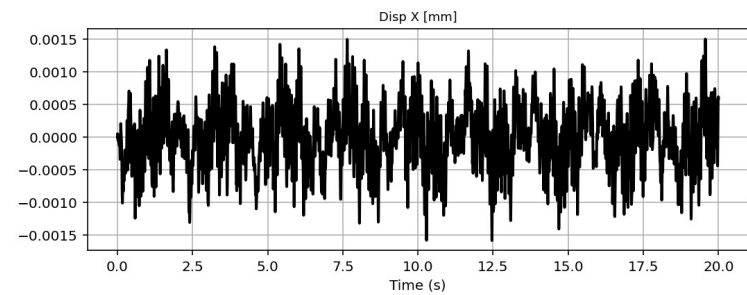
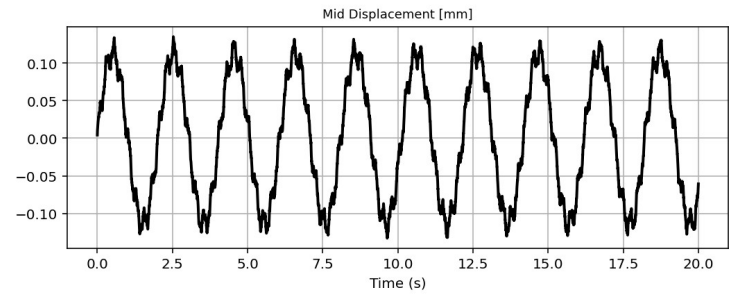
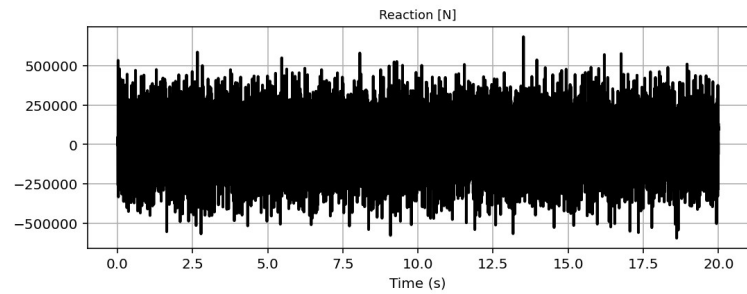


IPython Console Files Help Variable Explorer Debugger Plots History

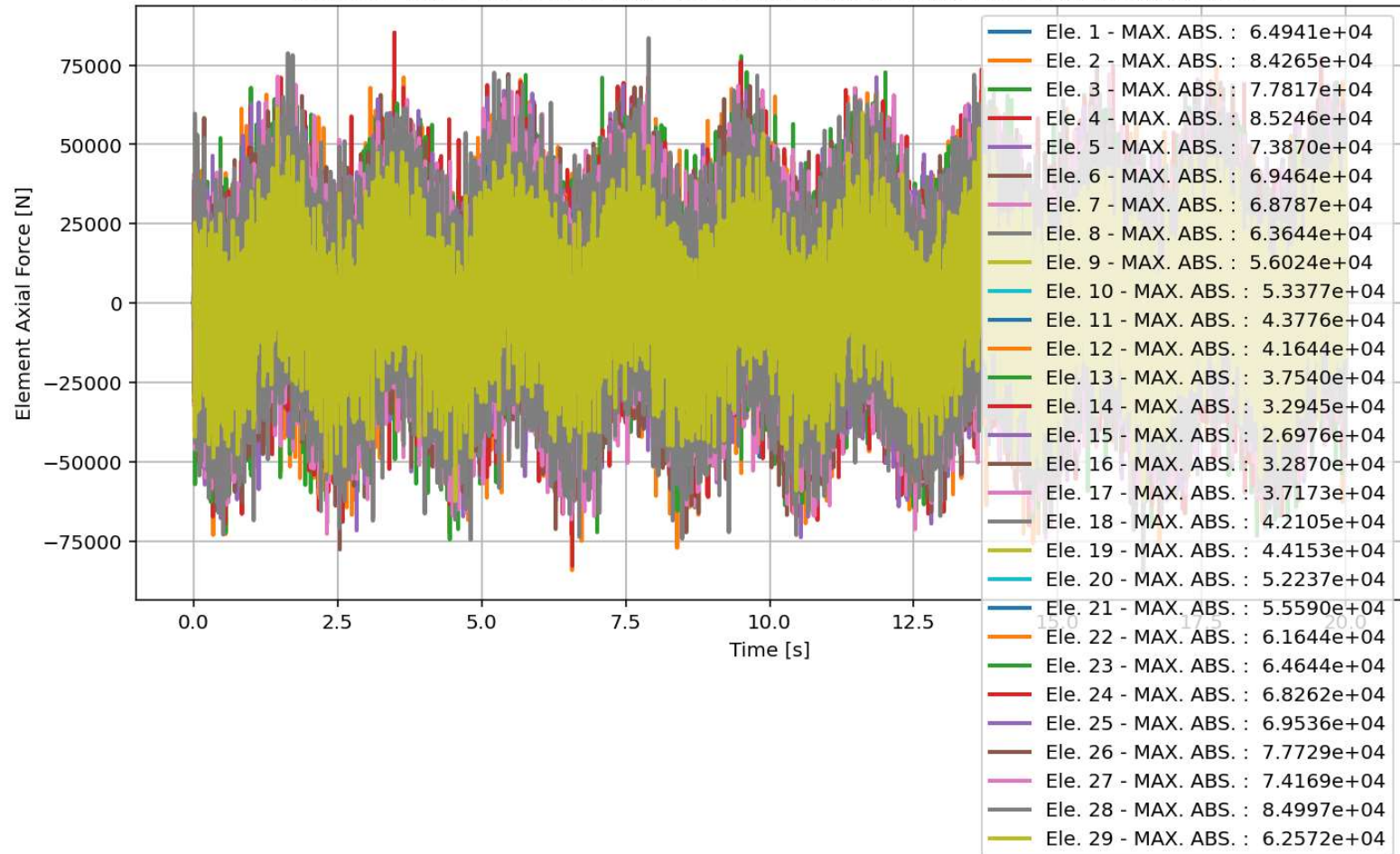
Conda: anaconda3 (Python 3.12.7) ✓ LSP: Python Line 699, Col 14 UTF-8 CRLF RW Mem 49%

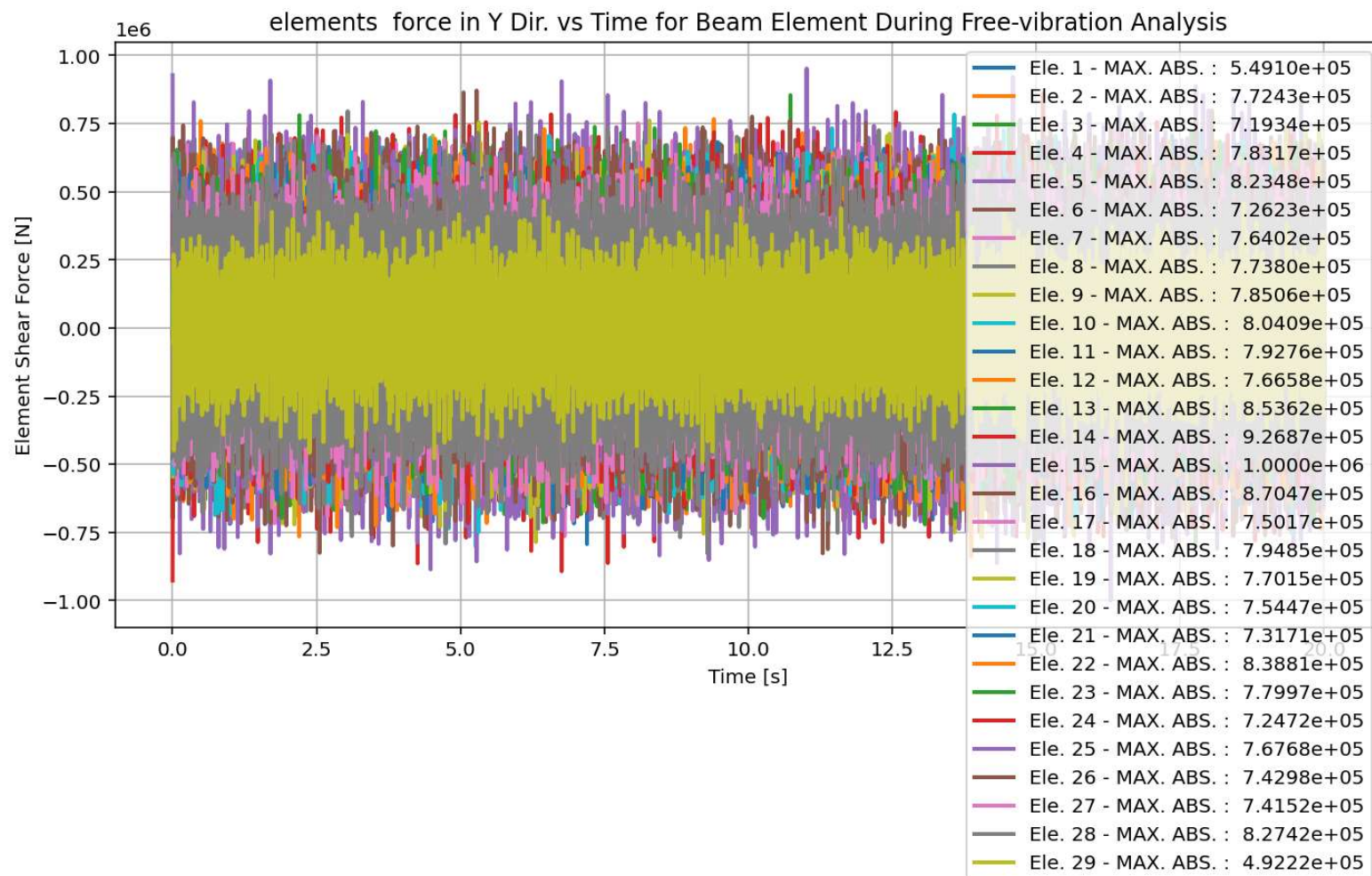
Base Reaction and Displacement of Structure During Free-vibration Analysis

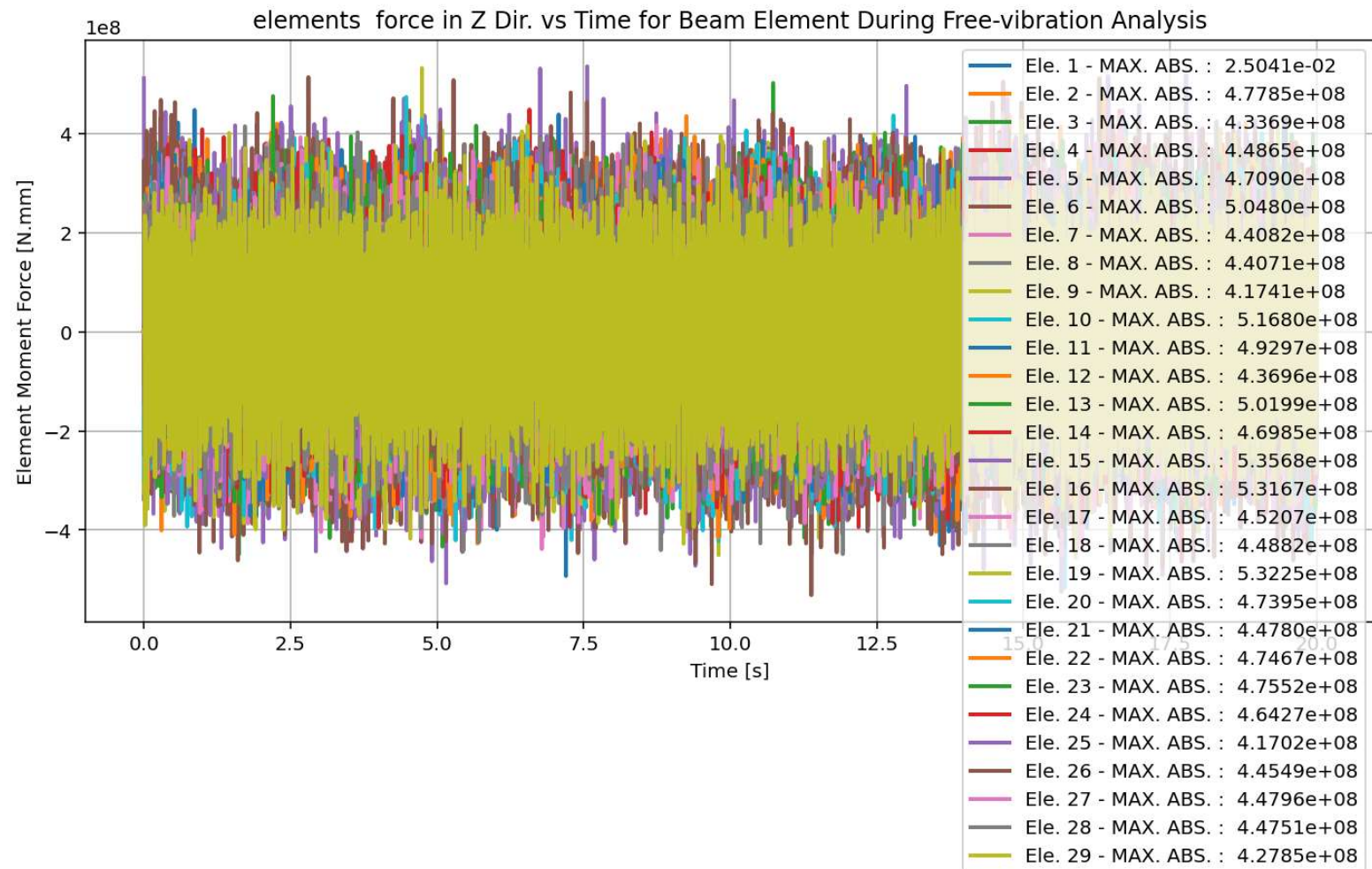




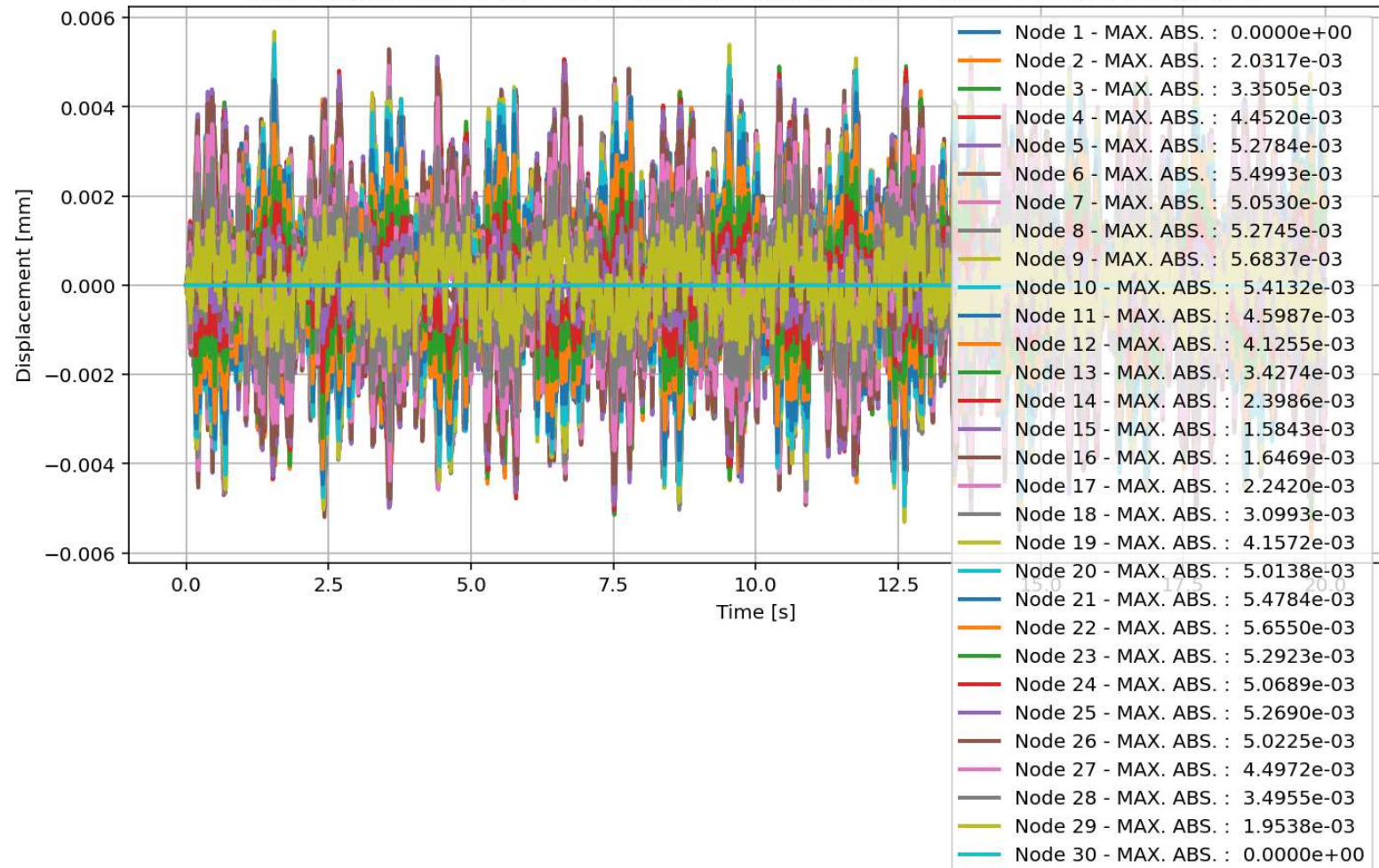
elements force in X Dir. vs Time for Beam Element During Free-vibration Analysis



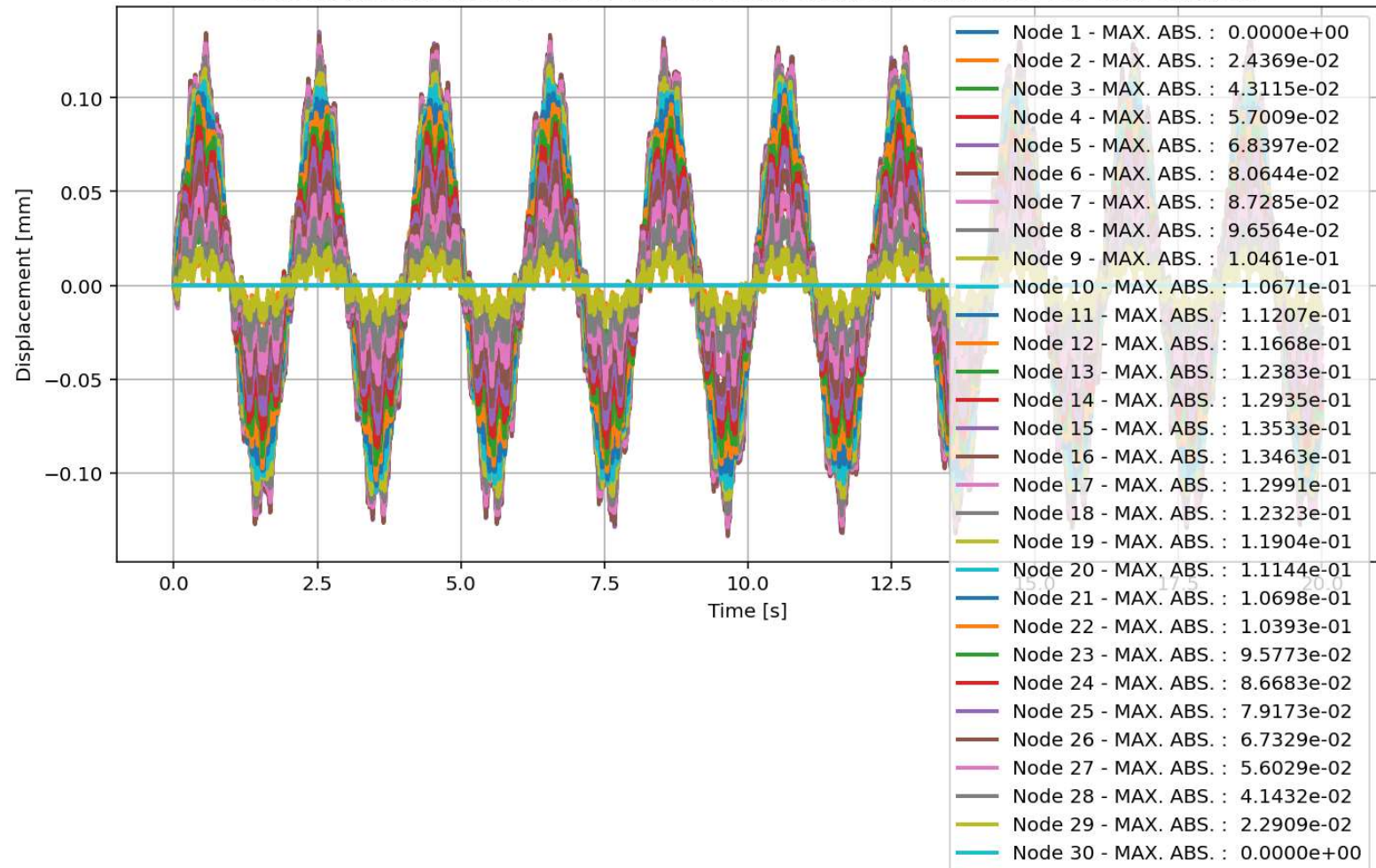


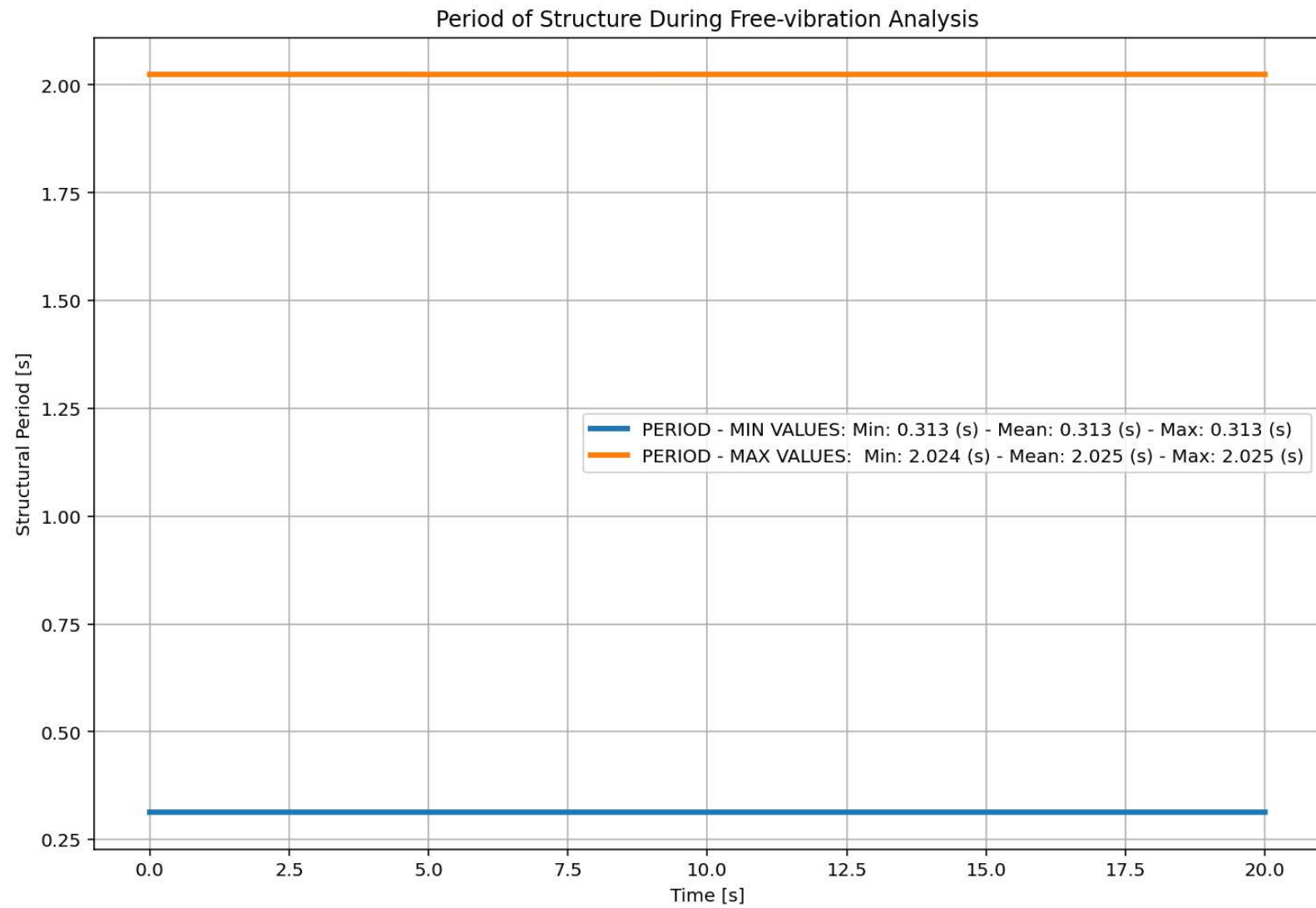


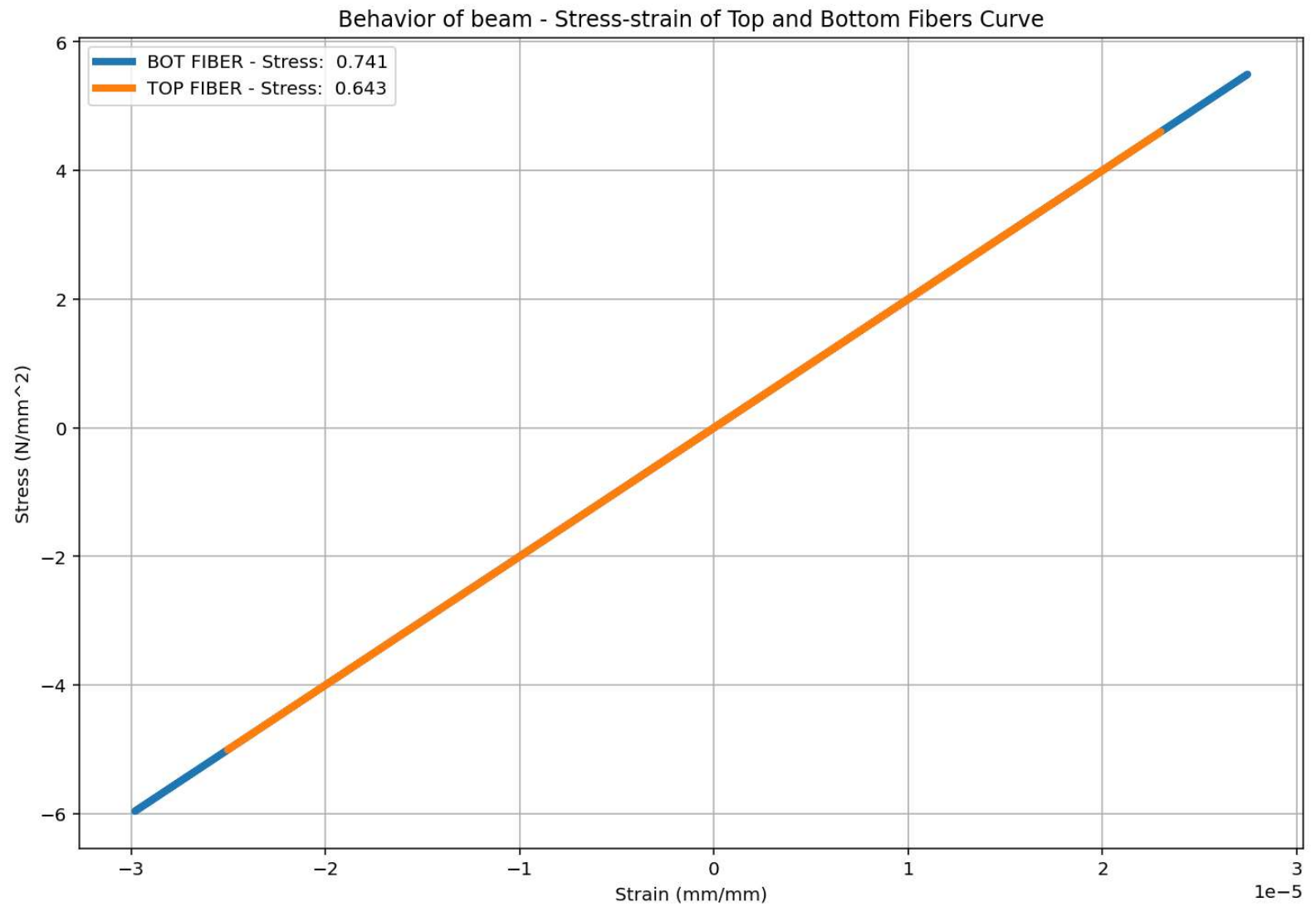
Node Displacements in X Dir. vs Time for Beam Element During Free-vibration Analysis

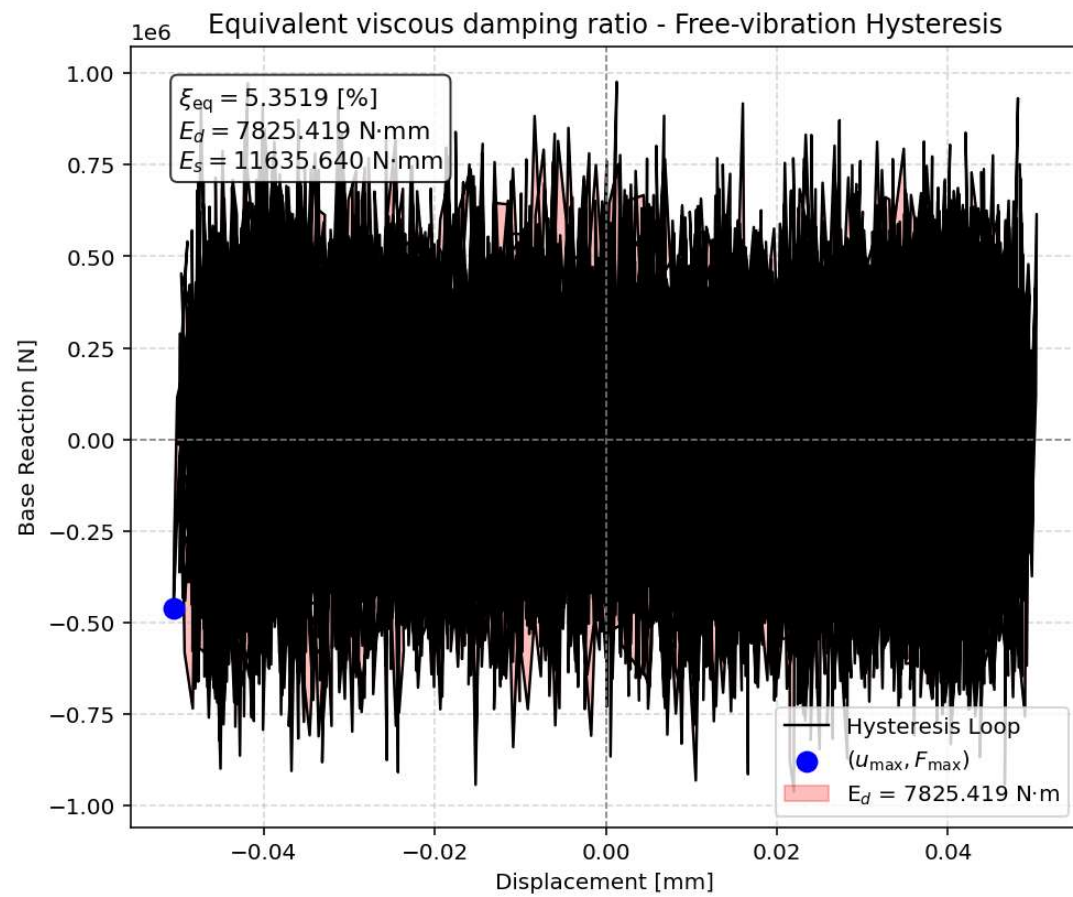


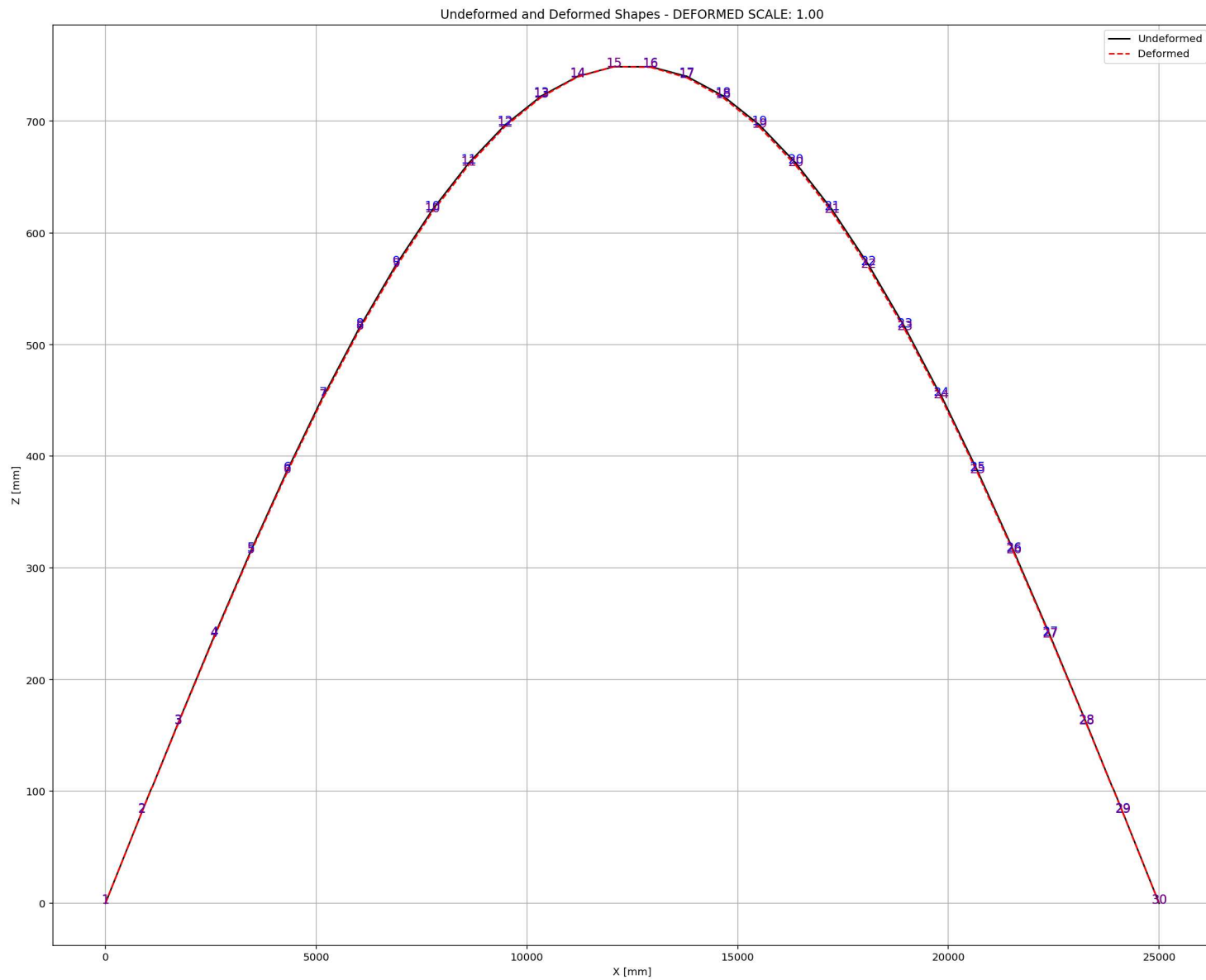
Node Displacements in Y Dir. vs Time for Beam Element During Free-vibration Analysis









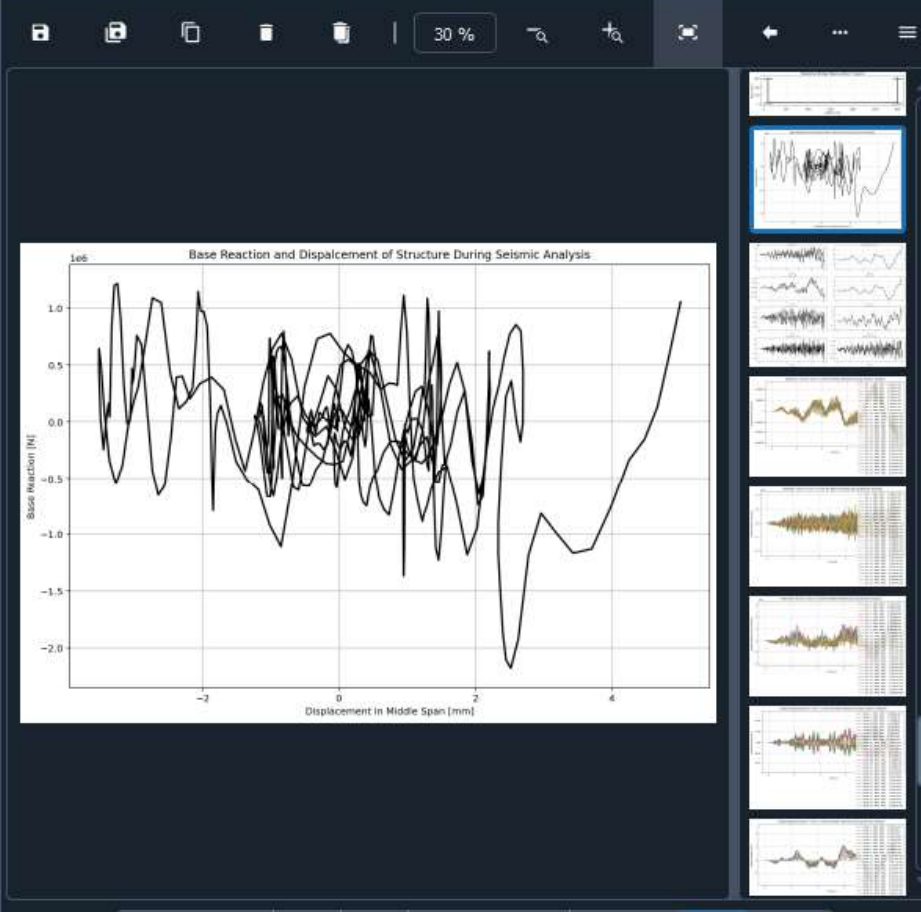


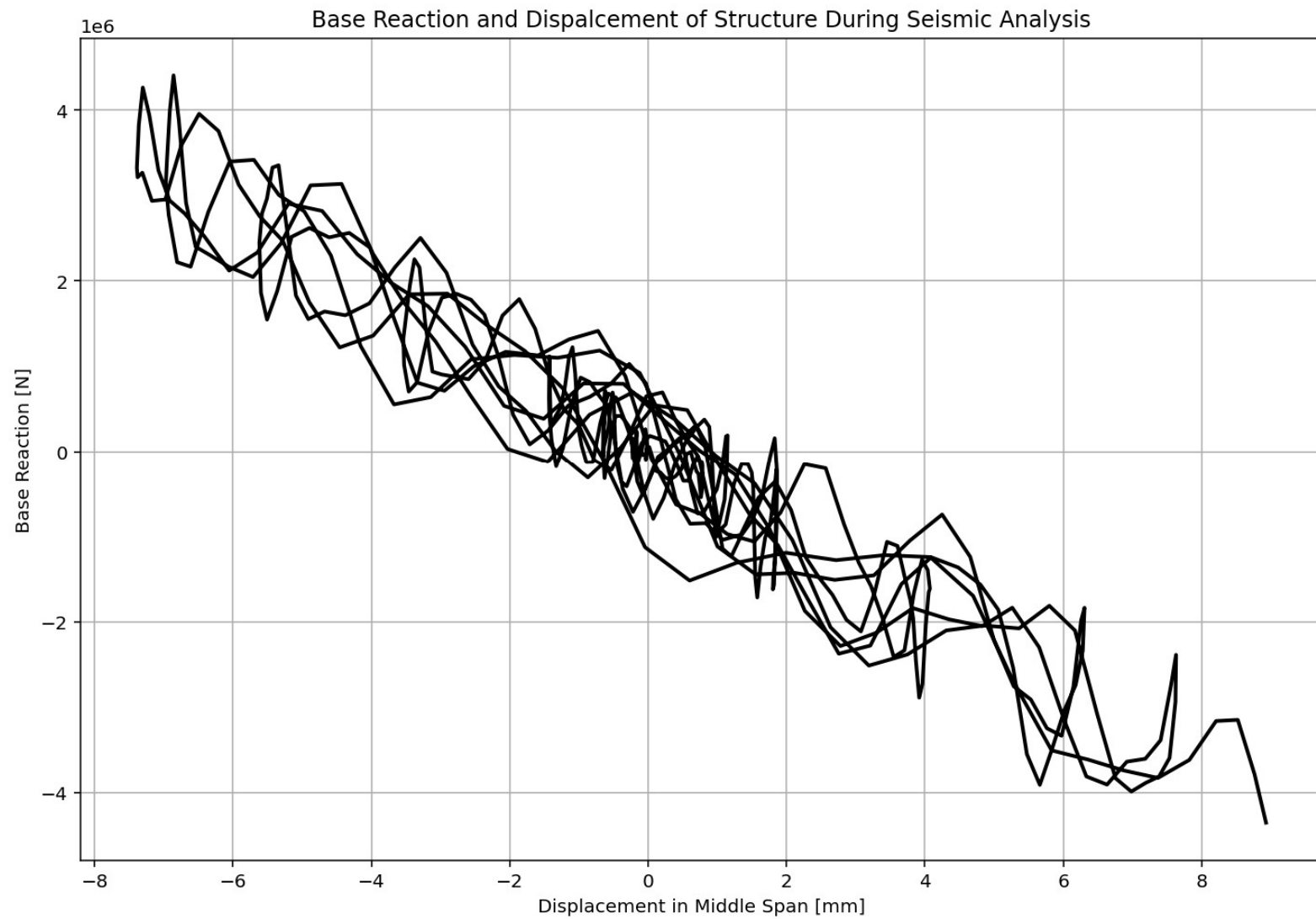
SEISMIC ANALYSIS

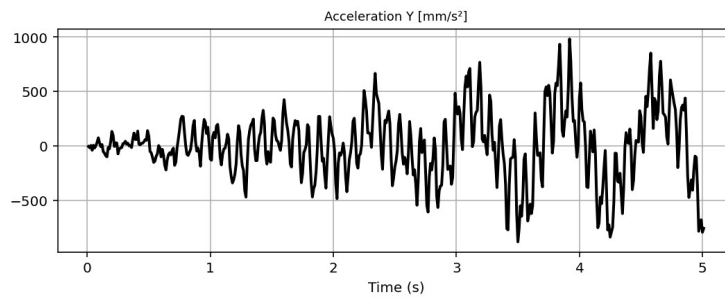
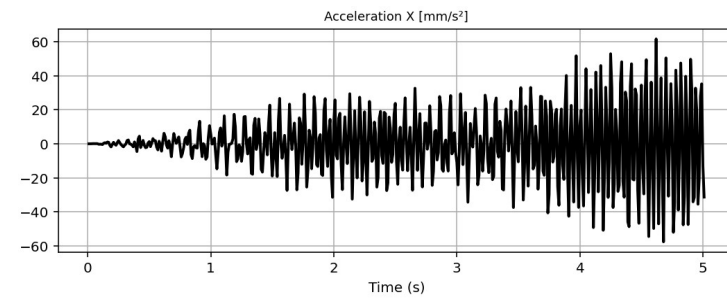
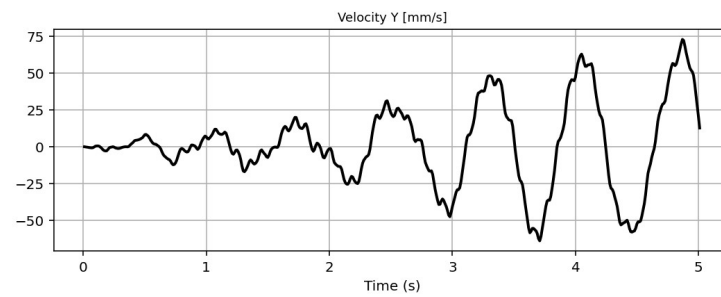
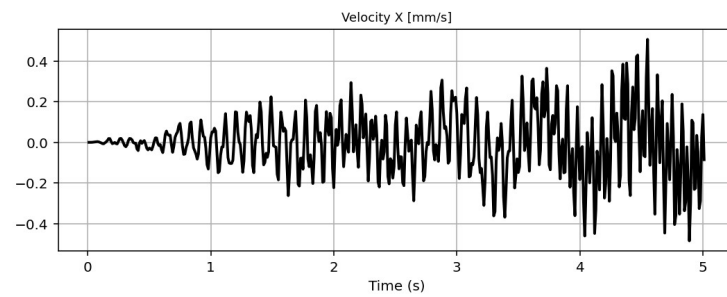
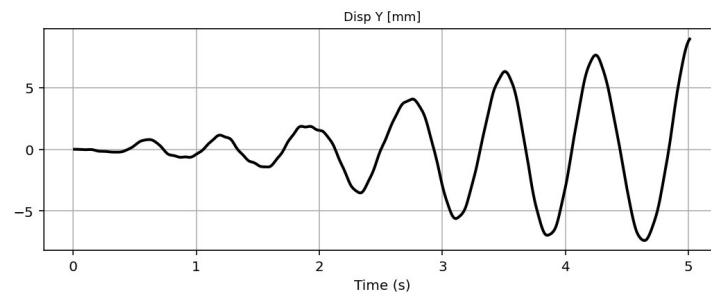
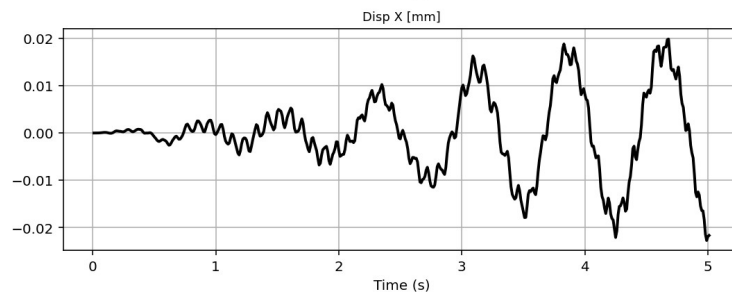
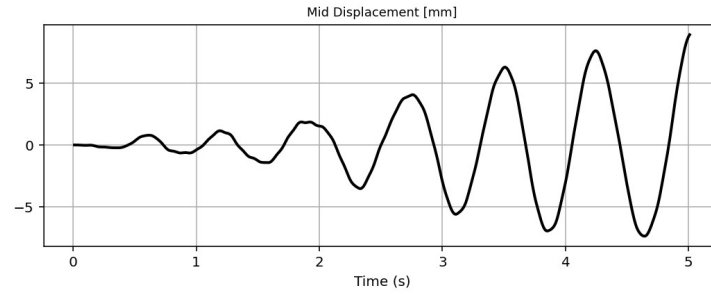
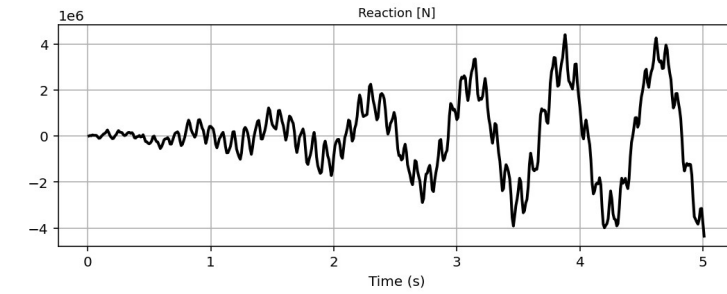
C:\Users\De\l\Desktop\OPENSEES_FILES\+EIGHT_ANALY...+P(t)_PEDESTRIAN_BRIDGE_STEEL_I_SECTION_8_ANA.py

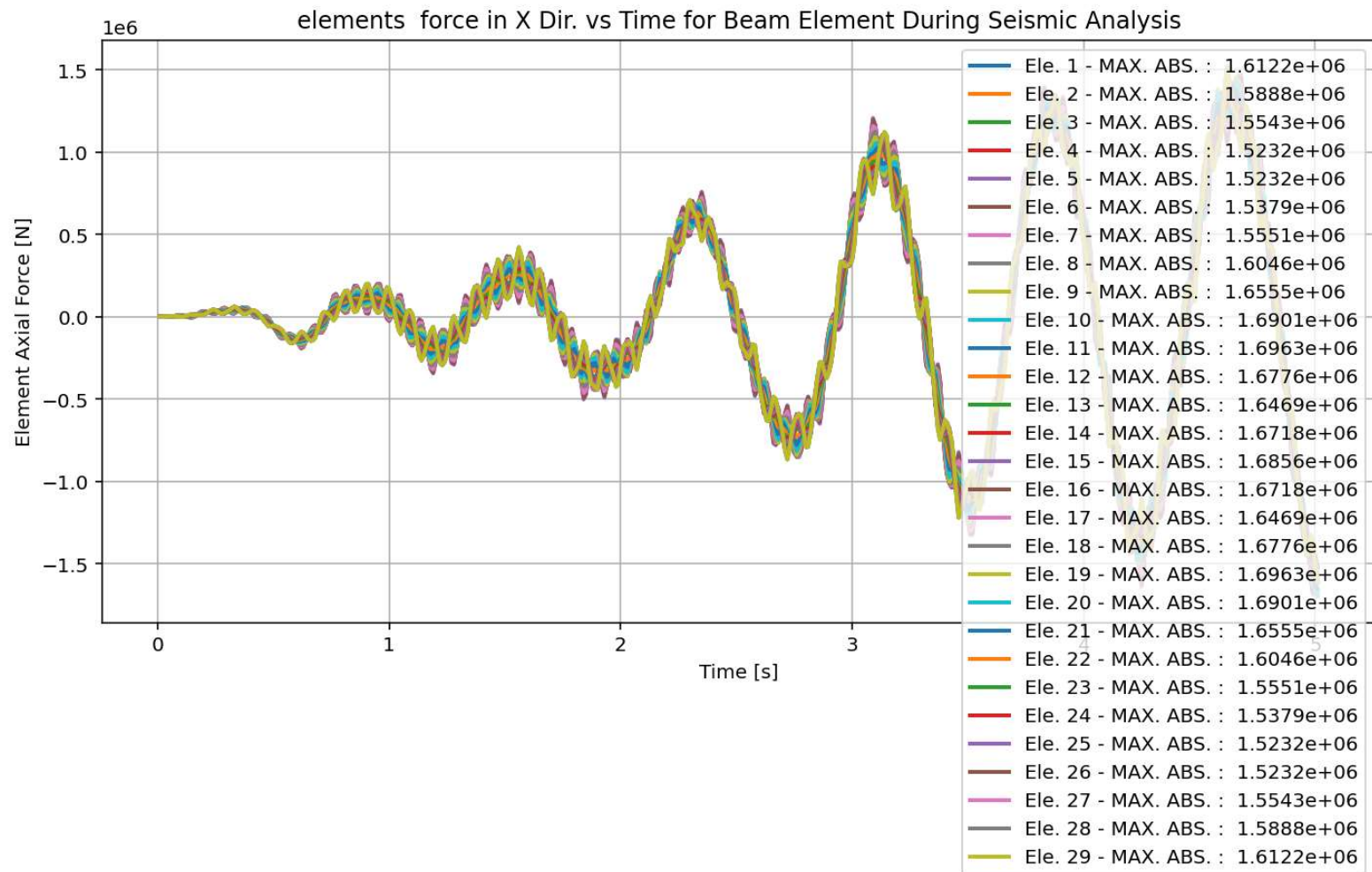
```

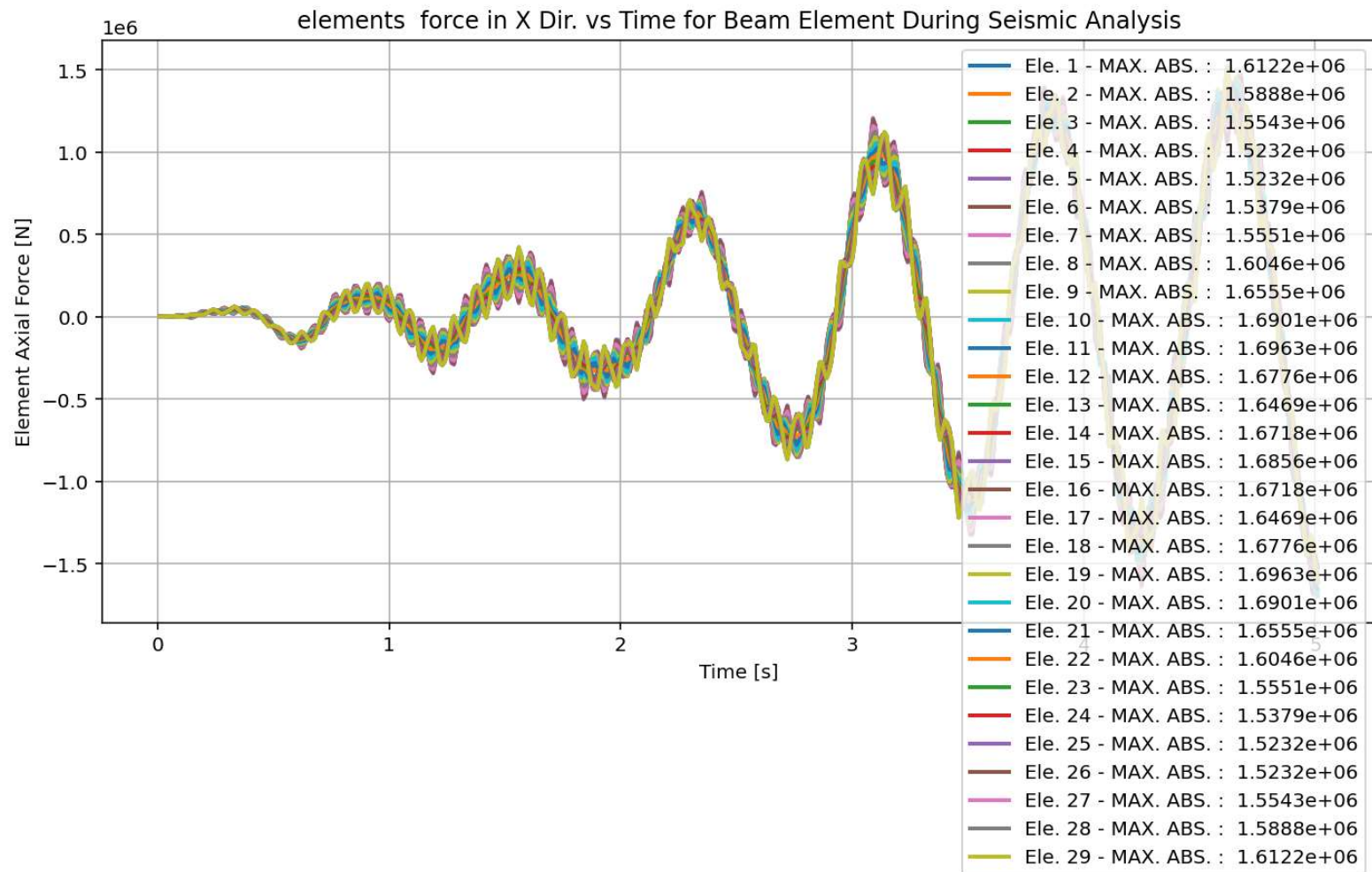
794
795 if ANAL_TYPE == 'SEISMIC': # DYNAMIC TIME-HISTORY ANALYSIS
796     %% DEFINE PARAMETERS FOR SEISMIC ANALYSIS
797     duration = 5.0           # [s] Analysis duration
798     dt = 0.01               # [s] Time step
799     GMfact = 9810           # [mm/s2] standard acceleration of gravity or st
800     SSF_X = 0.5             # Seismic Acceleration Scale Factor in X Direct
801     SSF_Y = 0.5             # Seismic Acceleration Scale Factor in Y Direct
802     iv0_X = 0.005           # [mm/s] Initial velocity applied to the node
803     iv0_Y = 0.005           # [mm/s] Initial velocity applied to the node
804     st_iv0 = 0.0            # [s] Initial velocity applied starting time
805     SEI = 'Y'               # Seismic Direction
806     DR = 0.03               # Damping ratio
807
808 # Define time series for input motion (Acceleration time history)
809 if SEI == 'X':
810     SEISMIC_TAG_01 = 100
811     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
812     # Define load patterns
813     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
814     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
815 if SEI == 'Y':
816     SEISMIC_TAG_02 = 200
817     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
818     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
819 if SEI == 'XY':
820     SEISMIC_TAG_01 = 100
821     ops.timeSeries('Path', SEISMIC_TAG_01, '-dt', dt, '-filePath', 'Ground_Acceler
822     # Define load patterns
823     # pattern UniformExcitation $patternTag $dof -accel $tsTag <-vel0 $vel0> <-fac
824     ops.pattern('UniformExcitation', SEISMIC_TAG_01, 1, '-accel', SEISMIC_TAG_01,
825     SEISMIC_TAG_02 = 200
826     ops.timeSeries('Path', SEISMIC_TAG_02, '-dt', dt, '-filePath', 'Ground_Acceler
827     ops.pattern('UniformExcitation', SEISMIC_TAG_02, 2, '-accel', SEISMIC_TAG_02,
    
```

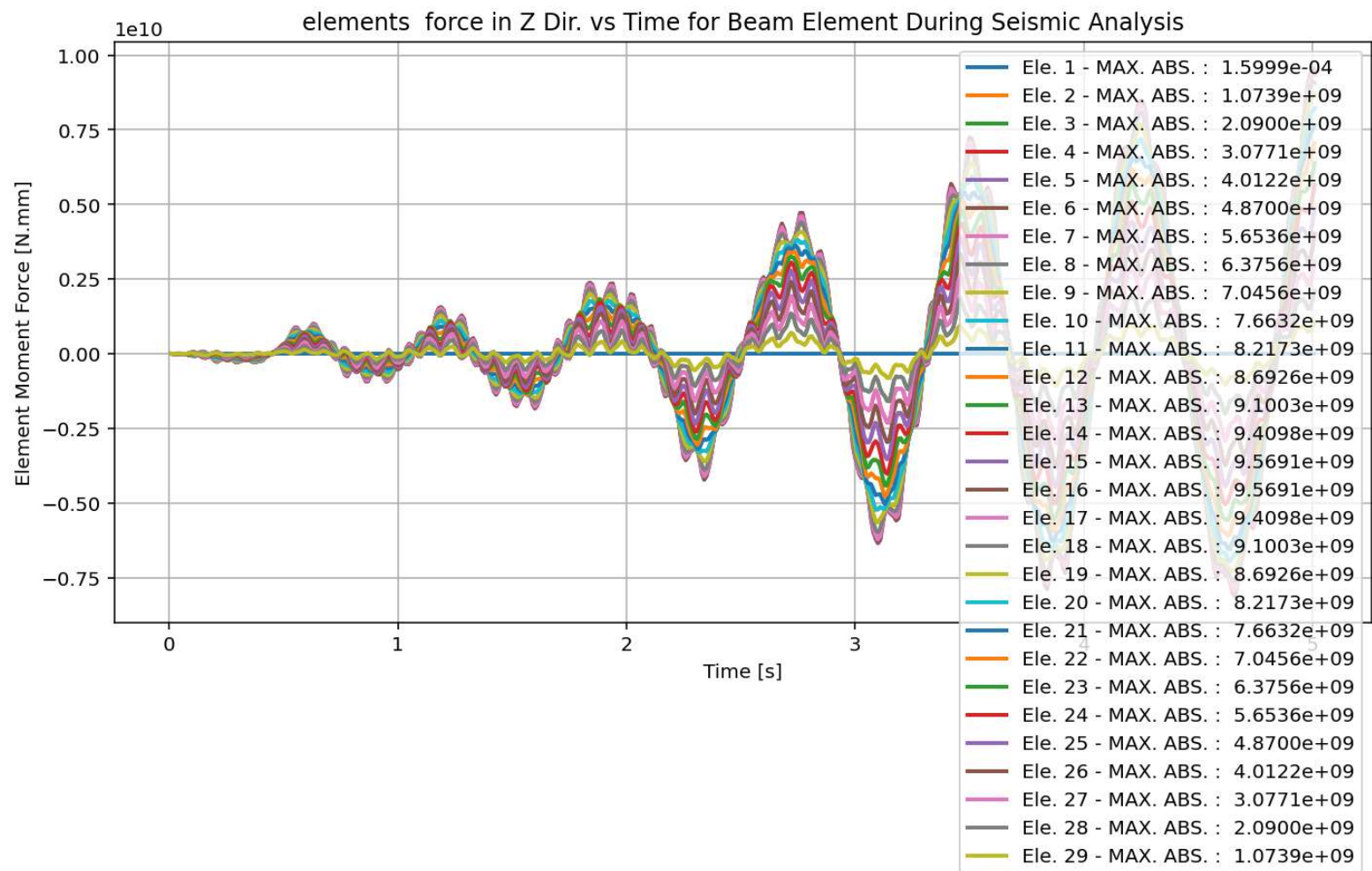




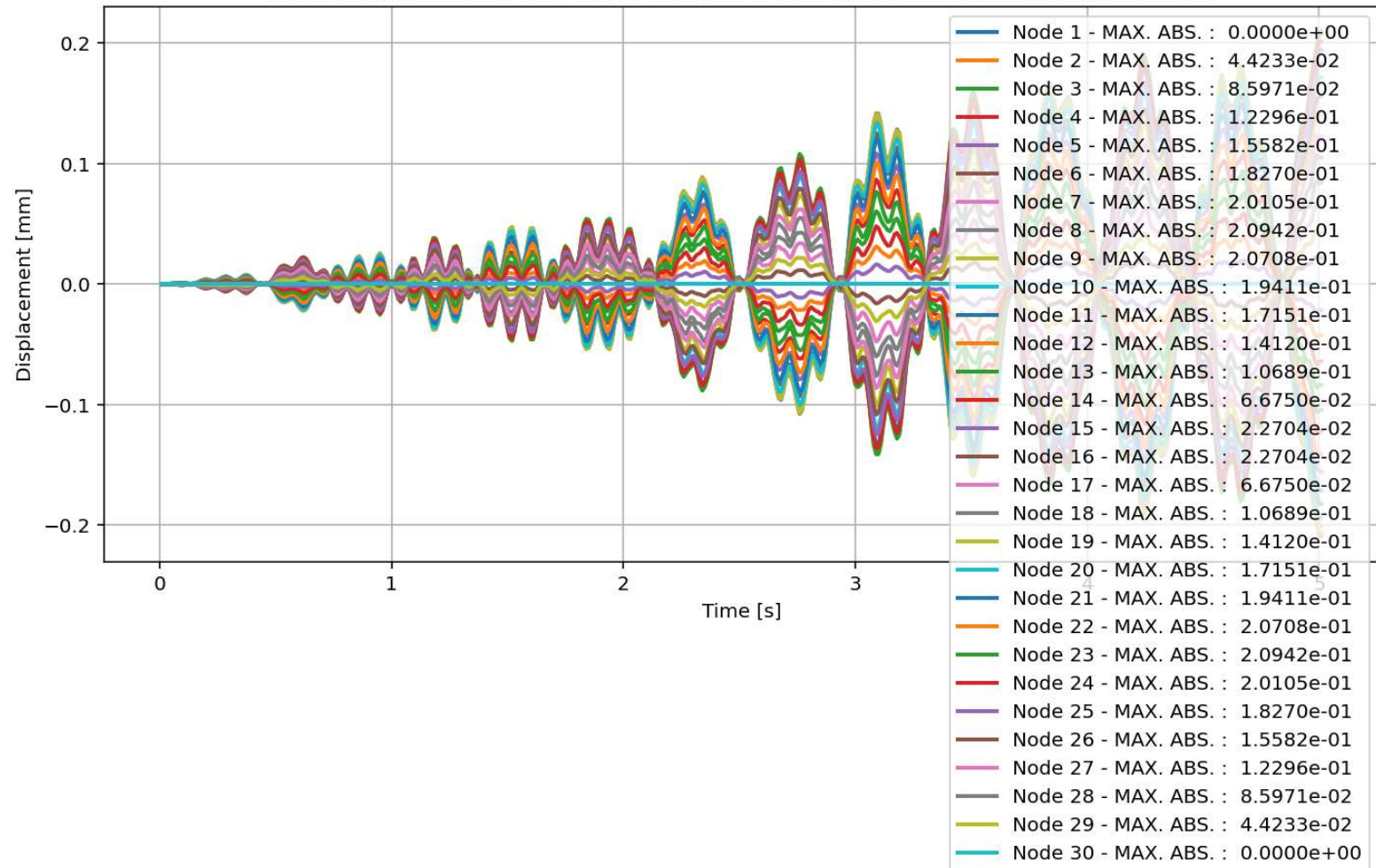




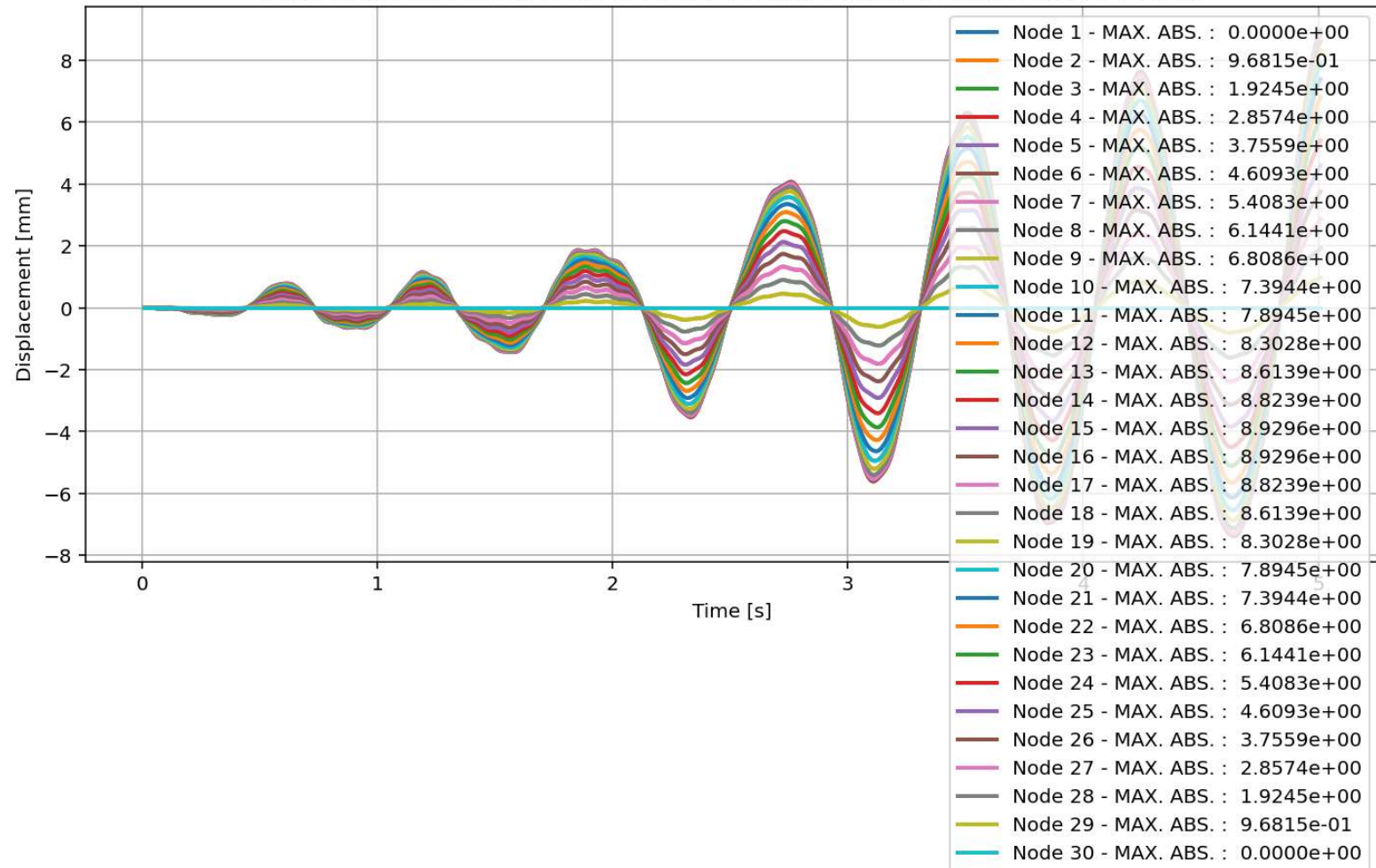


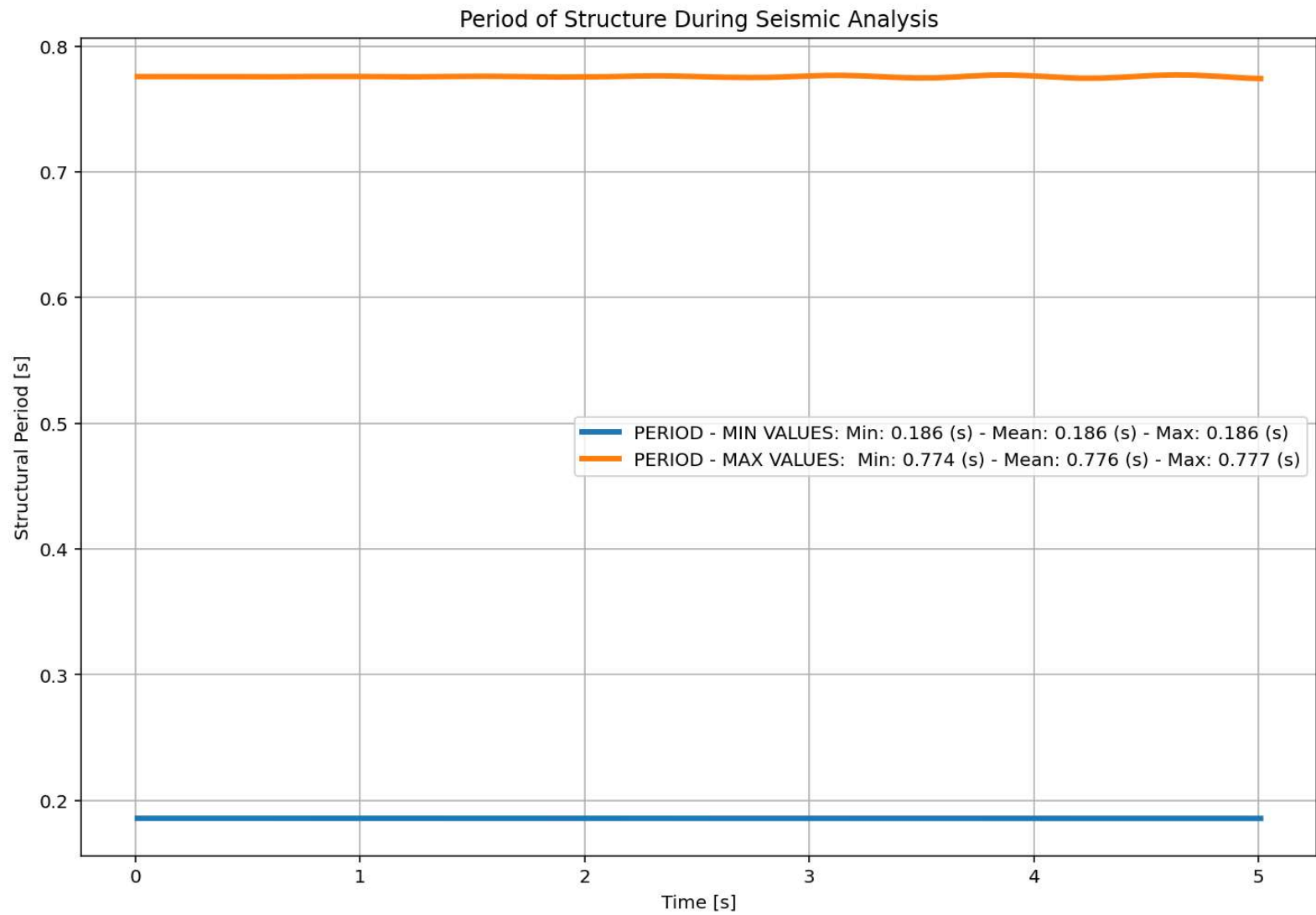


Node Displacements in X Dir. vs Time for Beam Element During Seismic Analysis

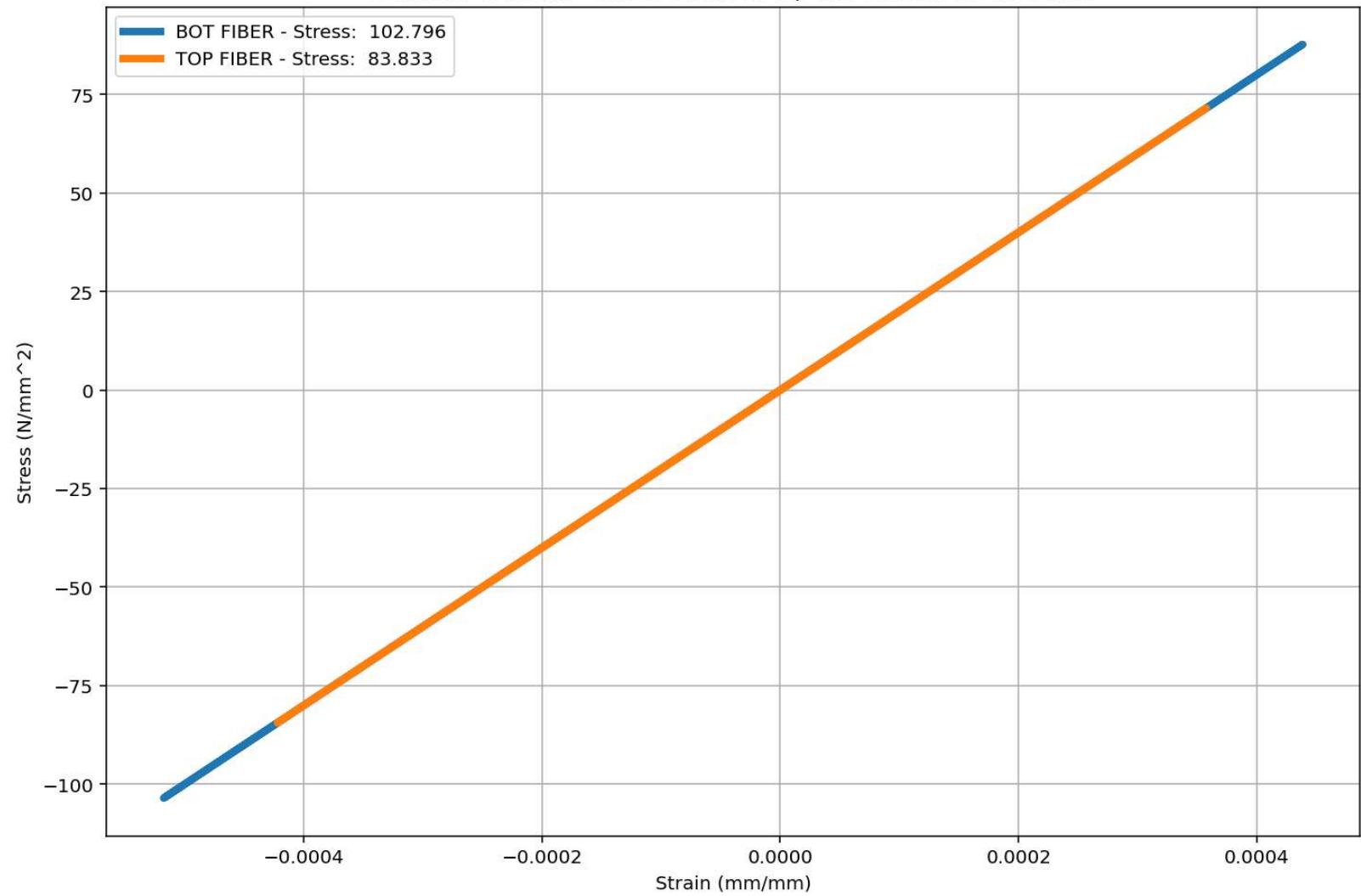


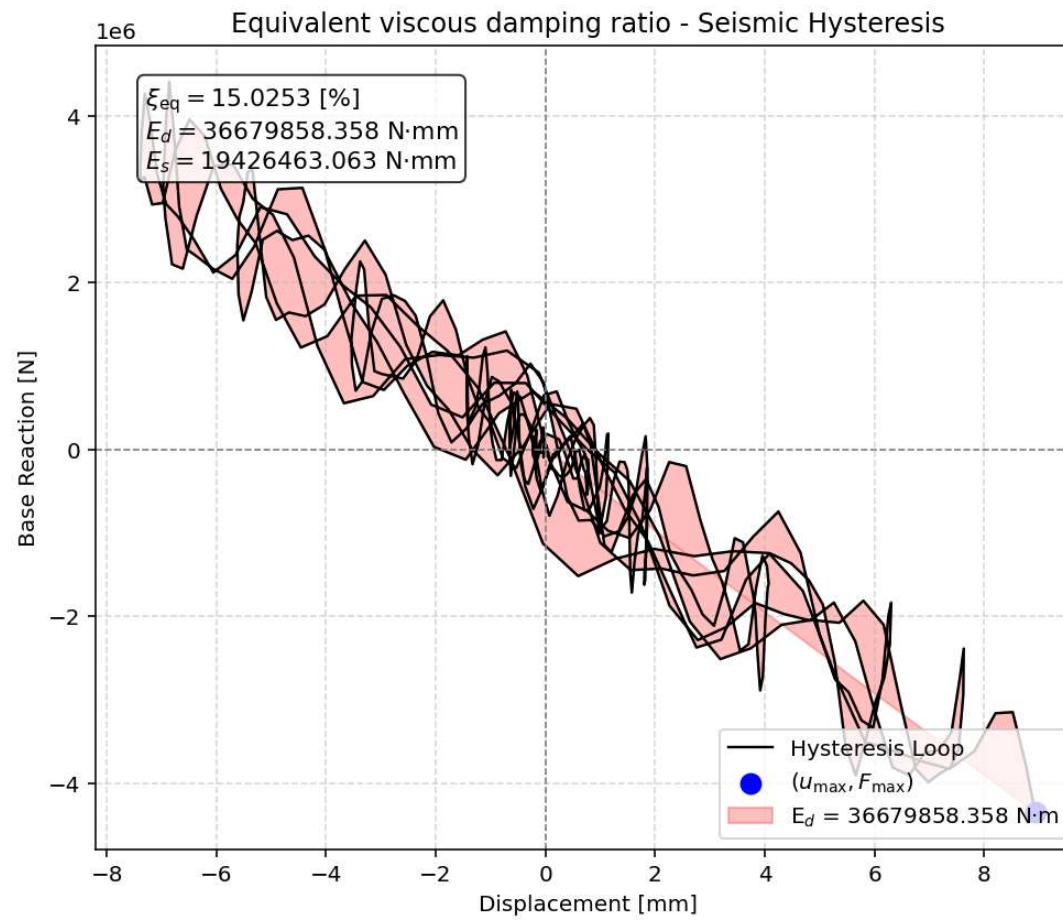
Node Displacements in Y Dir. vs Time for Beam Element During Seismic Analysis

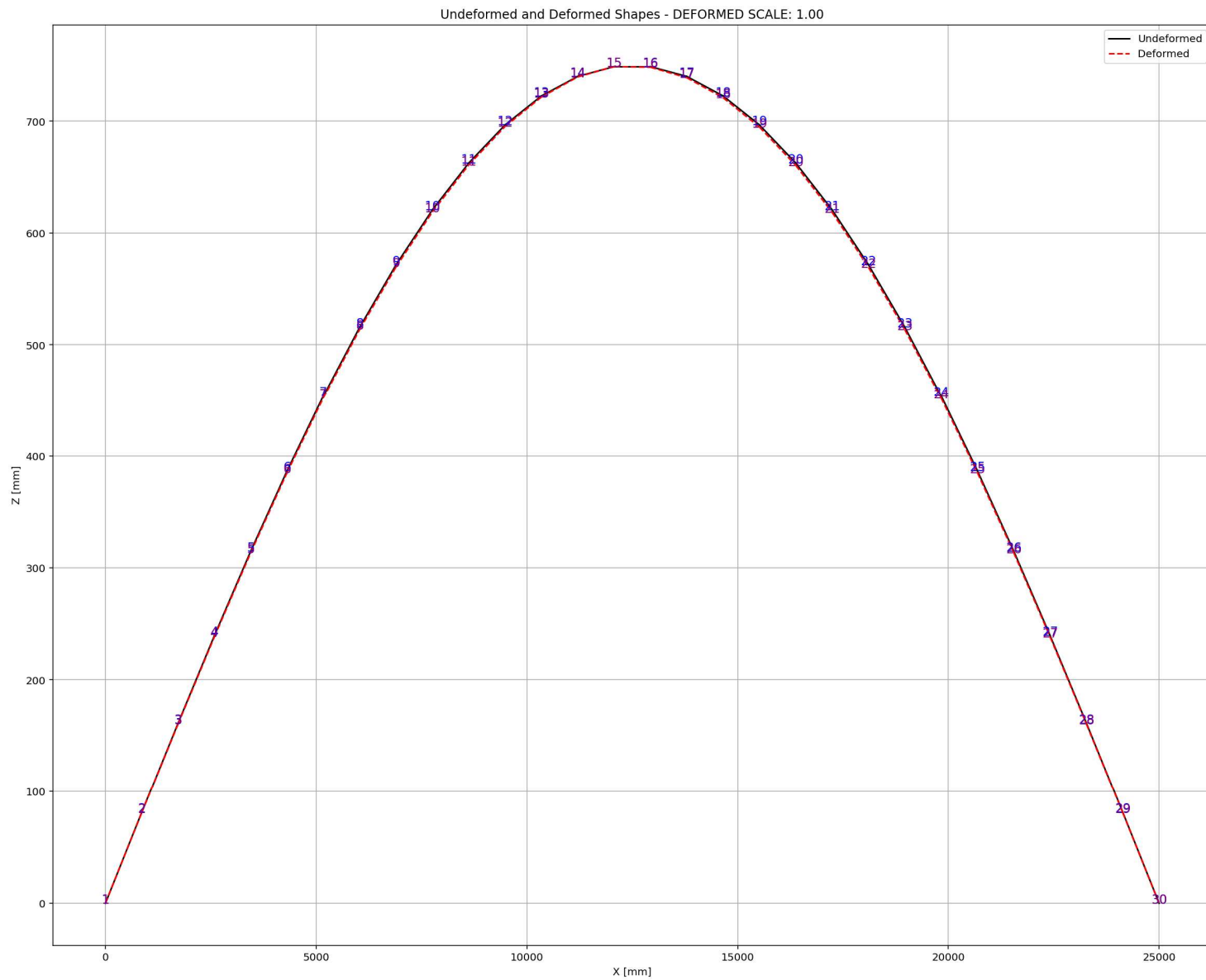


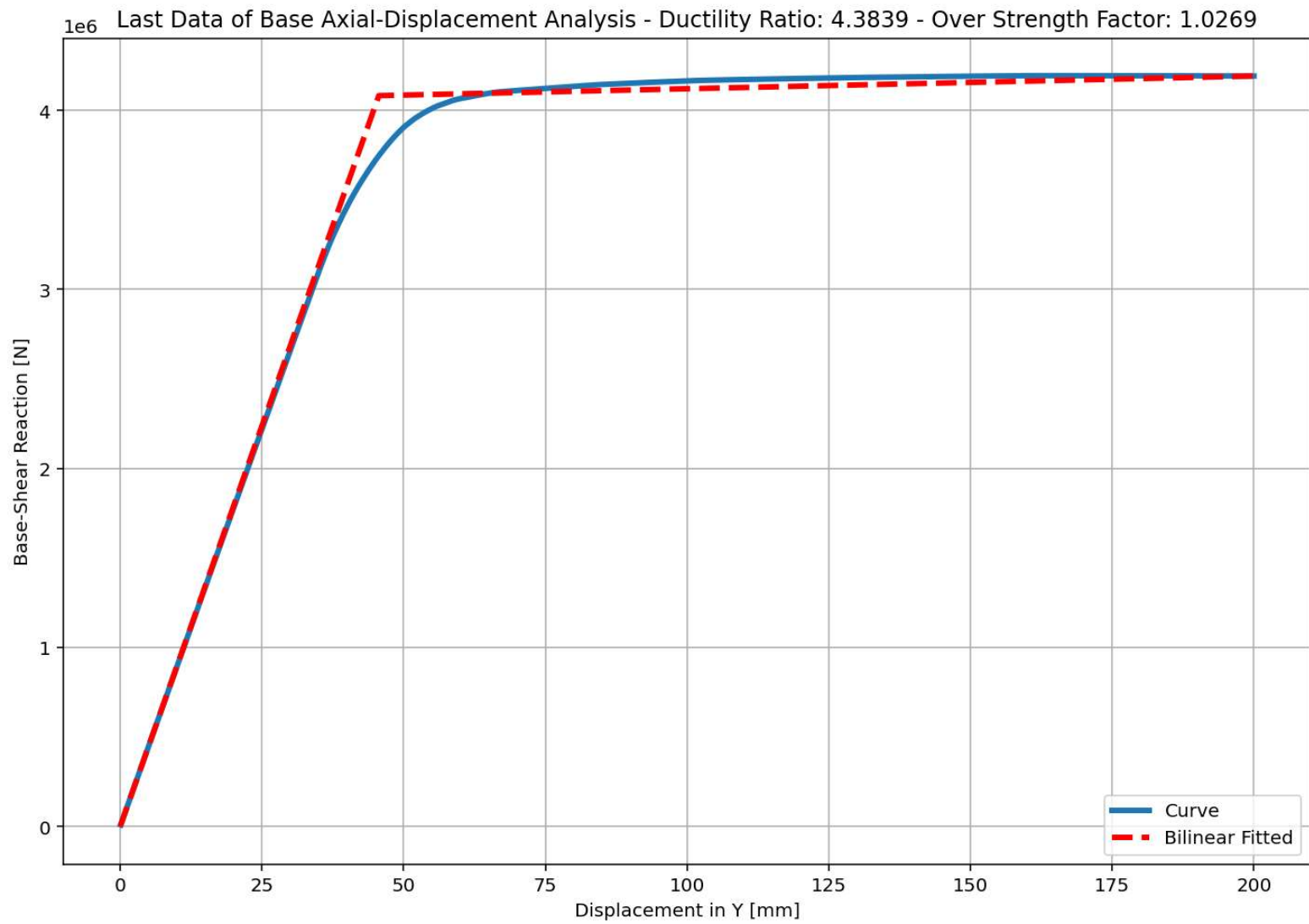


Behavior of beam - Stress-strain of Top and Bottom Fibers Curve









+=====+

= Analysis curve fitted =

Disp Base Shear

[[0.00000000e+00 0.00000000e+00]
[4.56219262e+01 4.08222187e+06]
[2.00000000e+02 4.19196500e+06]]

+=====+

+-----+

Structure Elastic Stiffness : 89479.38

Structure Plastic Stiffness : 20959.82

Structure Tangent Stiffness : 710.87

Structure Ductility Ratio : 4.38

Structure Over Strength Factor: 1.03

+-----+

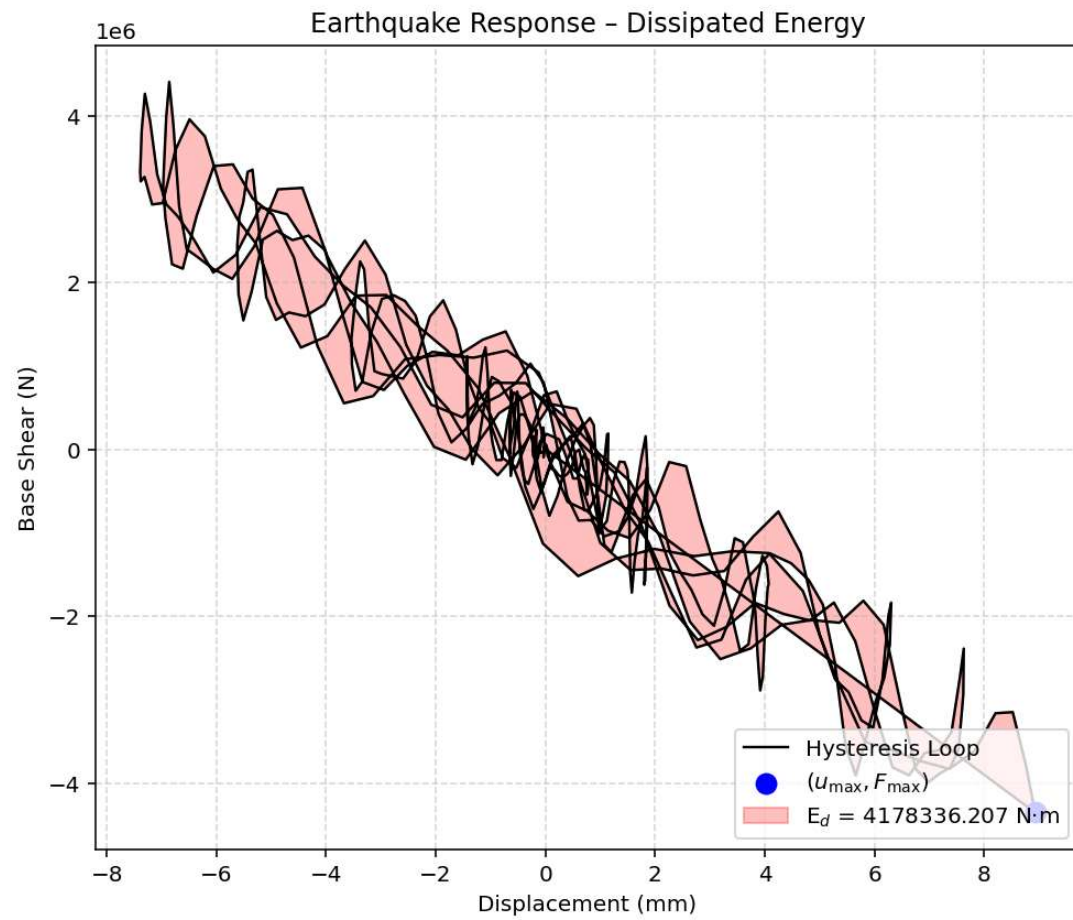
Over Strength Coefficient (Ω_0): 1.0269

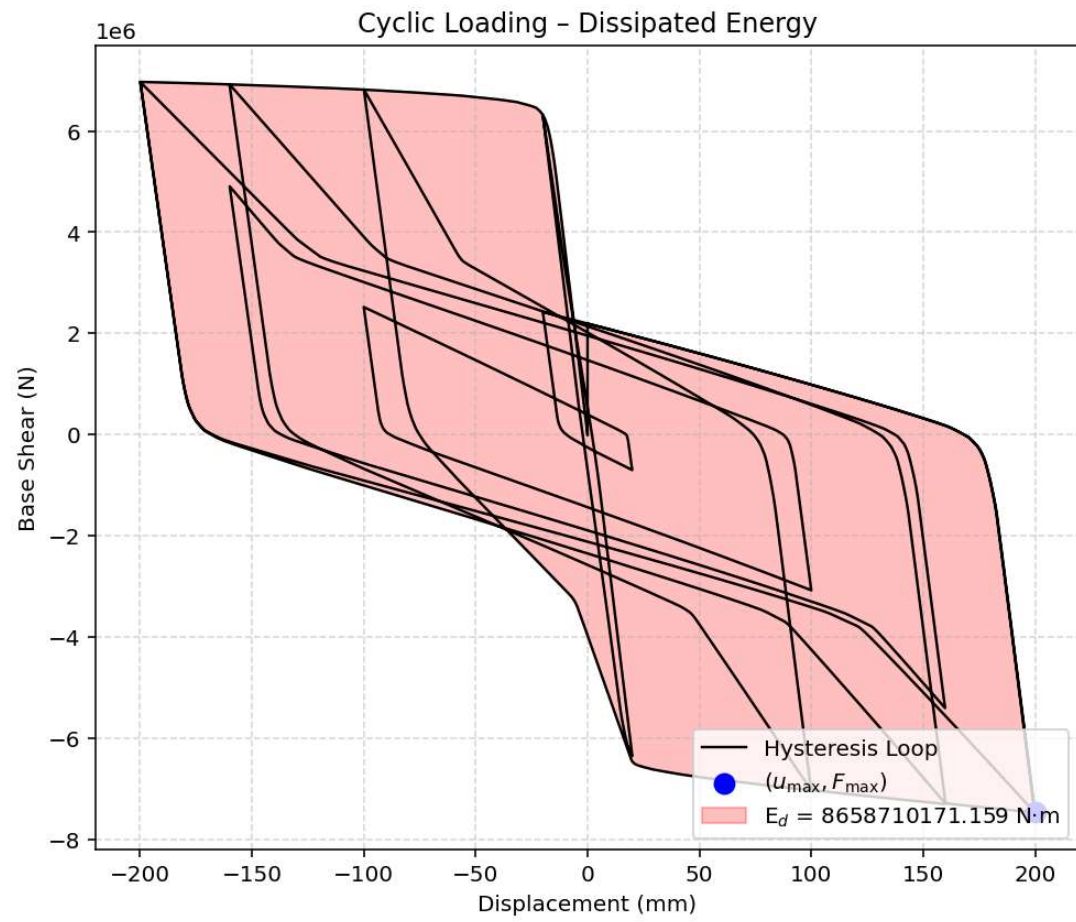
Displacement Ductility Ratio (μ): 4.3839

Ductility Coefficient (R_μ): 4.3839

Structural Behavior Coefficient (R): 4.5017

Structural Ductility Damage Index in Y Direction: -26.3043 (%)





Dissipated Energy from Earthquake= 4178336.21 N·mm

Dissipated Energy from Cyclic Displacement= 8658710171.16 N·mm

DISSIPATED ENERGY CAPACITY INDEX: 0.048 [%]

ZONE 1: VERY MINOR DAMAGE

